UNIVERSIDAD TECNOLÓGICA DE XICOTEPEC DE JUÁREZ

Ingeniería en Tecnologías de la Información e Innovación Digital
Optativa I: Bases de Datos en la Nube

Reporte técnico individual
Examen práctico Unidad 1
Implementación y administración de bases de datos en la nube

Estudiante	Jose Maria Castillo Aparicio
Matrícula	240628
Grupo / equipo	7A / Nexus Software
Docente	M.T.I. Marco A. Ramírez Hernández
Proveedor del VPS	Hostinger
IP del VPS	179.236.78.124
Puerto PostgreSQL/MySQL	40628/40629 (pgAdmin Web: 40630)
Repositorio / rama	https://github.com/Josemaria0304/Manual_VPS_BDs_Relacionales_240628

Fecha | 24 de octubre de 2026

Índice

Descripción de la práctica 3
Tecnologías utilizadas. 3
Desarrollo 4
Adquisición y configuración del VPS 4
Despliegue con Docker Compose 8
Puertos y conexiones remotas 11
Roles, usuarios y privilegios 14
Bases de datos y objetos relacionales 19
MySQL 19
PostgreSQL 22
Pruebas y resultados 23
Conclusiones 29
EVIDENCIAS GENERALES 31
Manual 36
Requerimientos para instalarlo 36
Pasos de Instalacion 36
Instalación y Configuración de Docker Compuse 47
Conclusión del Manual 54

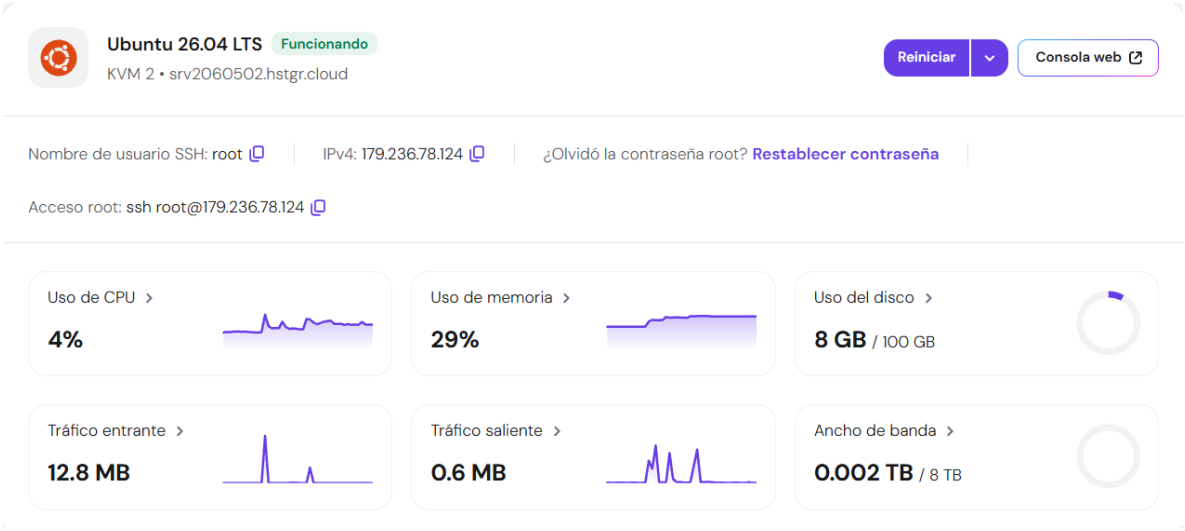
Descripción de la práctica

Contexto. La práctica consiste en desplegar y administrar servicios de bases de datos relacionales en un servidor privado virtual (VPS) compartido por el equipo, aplicados a un contexto de e-commerce (usuarios, productos, carritos, órdenes, pagos y bitácora de trazabilidad).

Objetivo. Desplegar PostgreSQL, MySQL Server y pgAdmin Web con Docker Compose de forma independiente a mis compañeros, y comprobar su funcionamiento: puertos, conexiones remotas, roles y privilegios, base de datos con tablas, triggers y vistas de trazabilidad, casos de prueba y control de versiones con Git.

Tecnologías utilizadas.

- VPS Hostinger compartido (dominio srv2060502.hstgr.cloud)
- Docker Engine y Docker Compose
- PostgreSQL 18 (versión estable actual)
- MySQL 8.4 (versión LTS)
- pgAdmin 4 Web (dpage/pgadmin4)
- Git y GitHub



Alcance. Despliegue con Compose, carga de la base de datos db_test en MySQL y su migración a PostgreSQL, y verificación de triggers y vistas. Los apartados de roles, casos de prueba y repositorio se documentan en las secciones correspondientes con su estado.

Servicio	Puerto publicado	Puerto interno	Criterio
PostgreSQL	40628	5432	Cinco dígitos de la matrícula
MySQL	40629	3306	Cinco dígitos de la matrícula + 1
pgAdmin Web	40630	80	Puerto libre elegido

Desarrollo

Adquisición y configuración del VPS

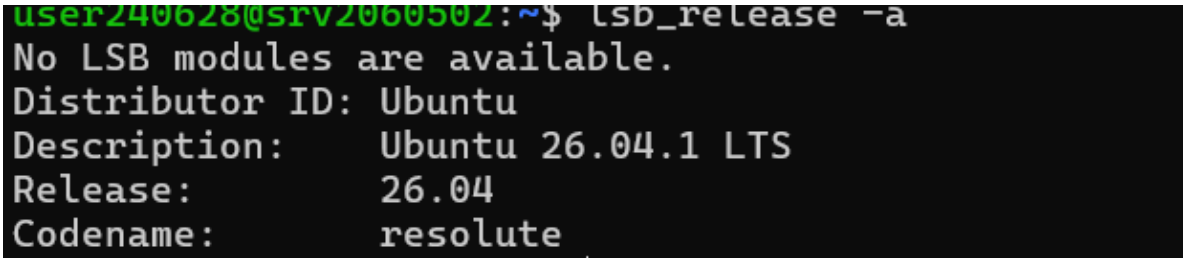
El VPS es un servidor compartido de Hostinger con el dominio srv2060502.hstgr.cloud, accesible por SSH desde Internet. Mi cuenta personal de sistema operativo es user240628 (uid 1000), que pertenece a los grupos user240628, users y docker.

Al intentar ejecutar sudo el sistema lo rechazó, y el comando id confirmó que mi usuario no pertenece al grupo sudo. Esto limita la administración a nivel de sistema

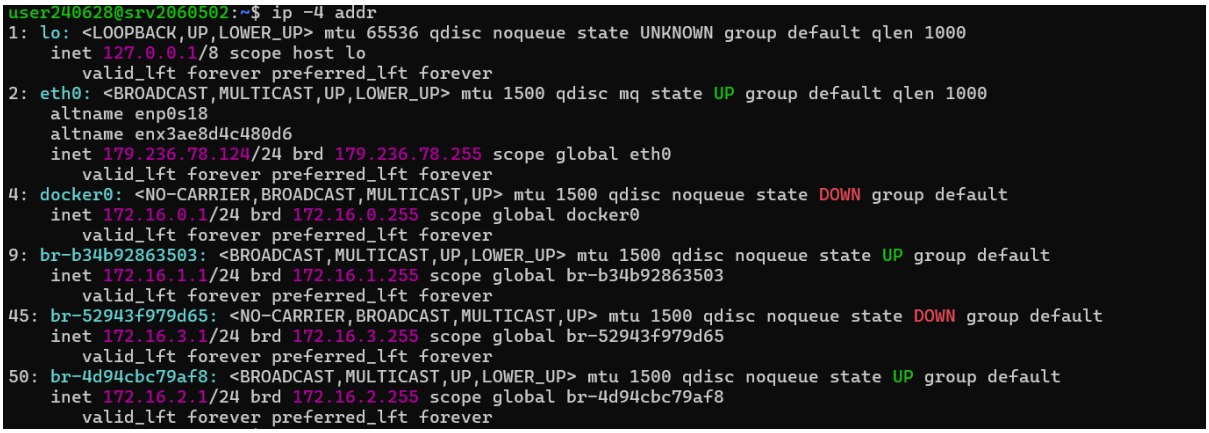
(por ejemplo, la configuración de ufw) hasta que el administrador del VPS asigne el permiso.

Comandos para documentar la configuración general:

lsb_release -a: muestra la distribución de Linux instalada (nombre, versión y nombre en clave). La opción -a pide toda la información. Sirve para documentar el sistema operativo del VPS.



ip -4 addr: lista las interfaces de red y sus direcciones IPv4 (-4 oculta las IPv6). Cada interfaz muestra su dirección en el campo inet. Es la forma de comprobar la IP del VPS.



nproc: imprime cuántos procesadores (núcleos) puede usar el sistema.



free -h: muestra la memoria RAM y el swap. La opción -h los presenta en unidades legibles (Gi, Mi). La columna que importa es available, la memoria que se puede usar sin problemas, porque free incluye la que el sistema usa como caché y la libera

cuando hace falta.

```
user240628@srv2060502:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.7Gi       2.6Gi       439Mi        46Mi       5.1Gi       5.2Gi
Swap:             0B          0B          0B
```

df -h: muestra el espacio de los discos y particiones: tamaño, usado, disponible y punto de montaje. Con -h también sale en unidades legibles.

```
user240628@srv2060502:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           1.6G  1.3M  1.6G   1% /run
/dev/sda1        96G  8.4G   88G   9% /
tmpfs           3.9G     0  3.9G   0% /dev/shm
tmpfs           3.9G     0  3.9G   0% /tmp
/dev/sda13      989M  150M  773M  17% /boot
/dev/sda15      105M  6.3M   99M   7% /boot/efi
none            1.0M     0  1.0M   0% /run/credentials/getty@tty1.service
none            1.0M     0  1.0M   0% /run/credentials/serial-getty@ttyS0.service
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
none            1.0M     0  1.0M   0% /run/credentials/systemd-networkd.service
tmpfs           794M  4.0K  794M   1% /run/user/1000
```

Id

```
user240628@srv2060502:~$ id
uid=1000(user240628) gid=1000(user240628) groups=1000(user240628),100(users),986(docker)
```

Capturas:

- **IP pública:** 179.236.78.124, en la interfaz eth0. Esa es la dirección con la que se llega al VPS desde Internet. La interfaz lo (127.0.0.1) es la interna del propio equipo.
- **Procesador:** 2 núcleos.
- **Memoria:** 7.7 GiB en total, de los cuales hay 5.2 GiB disponibles. Ahora se usan 2.6 GiB, que incluyen los servicios de mis compañeros. No tiene swap (0B), así que si alguien consume toda la RAM, el sistema puede cerrar procesos a la fuerza.
- **Disco:** la partición principal /dev/sda1 tiene 96 GB, con 8.4 GB usados (9%) y 88 GB libres. Las demás líneas son particiones de arranque y sistemas temporales en memoria.
- **Usuario:** uid=1000, grupos user240628, users y docker. Confirma que puedo usar Docker y que **no estoy en un grupo sudo**.
- **Redes de Docker:** además de docker0 aparecen tres interfaces br-… con direcciones 172.16.x.1. Son redes que Docker Compose crea para cada

proyecto. Las que están en UP tienen contenedores conectados (mios y de mis compañeros), y las DOWN están sin uso.

Cuentas y permisos del sistema. Para comprobar las cuentas personales y los permisos de administración se usan estos comandos, que no requieren sudo porque las listas de cuentas y grupos son de lectura pública:

Comando	Qué hace
ls /home	Lista las carpetas personales, una por cada cuenta del sistema.
getent passwd | awk ...	Muestra solo las cuentas de personas (uid de 1000 en adelante): usuario, número y carpeta personal.
getent group sudo y getent group wheel	Muestran los miembros con permisos de administrador. El grupo se llama sudo o wheel según la distribución.
getent group docker	Muestra quién puede crear y borrar contenedores, un permiso casi administrativo.
id nombre_del_usuario	Muestra los grupos de un integrante concreto del equipo.

ls /home

getent passwd | awk -F: '$3>=1000 && $3<60000 {print $1, $3, $6}'

```
user240628@srv2060502:~$ ls /home
getent passwd | awk -F: '$3>=1000 && $3<60000 {print $1, $3, $6}'
user240161  user240165  user240201  user240628  user240763  user240808
user240628 1000 /home/user240628
user240808 1001 /home/user240808
user240763 1002 /home/user240763
user240161 1003 /home/user240161
user240165 1004 /home/user240165
user240201 1005 /home/user240201
```

getent group sudo

```
user240628@srv2060502:~$ getent group sudo
sudo:x:27:
```

getent group Wheel

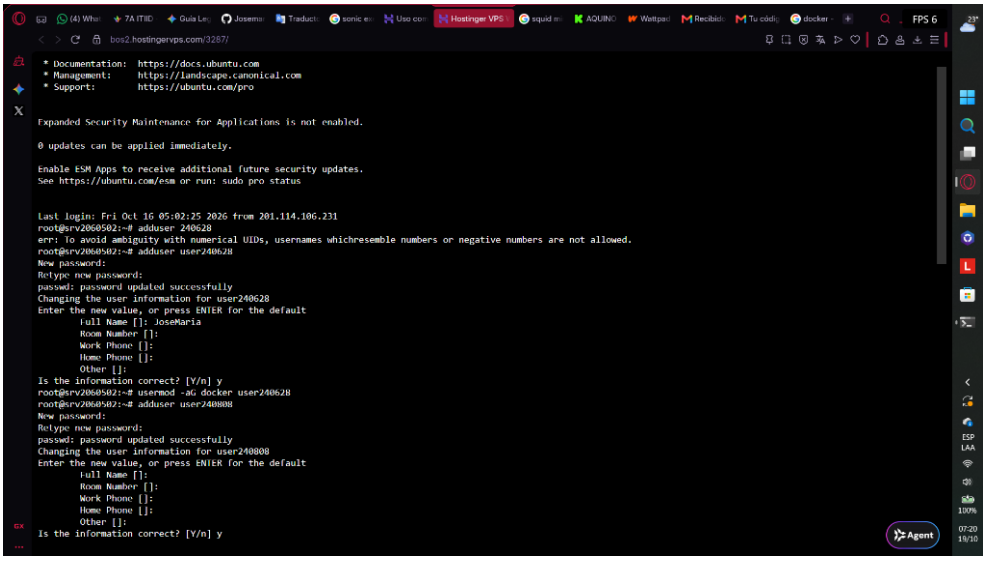
```
user240628@srv2060502:~$ getent group wheel
```

getent group docker

```
user240628@srv2060502:~$ getent group docker
docker:x:986:user240628,user240808,user240763,user240161,user240165,user240201
```

Mi usuario pertenece al grupo docker, pero no al grupo sudo, por lo que puedo manejar contenedores y no administrar el sistema.

Cuentas de los integrantes. Durante el desarrollo se crearon en el VPS las cuentas personales de acceso de mis compañeros con el comando sudo adduser nombre_usuario, que crea la cuenta, su grupo y su carpeta personal en /home, y pide la contraseña inicial. Después se asignaron los grupos según las responsabilidades de cada integrante: solo se maneja el docker para los contenedores.



Despliegue con Docker Compose

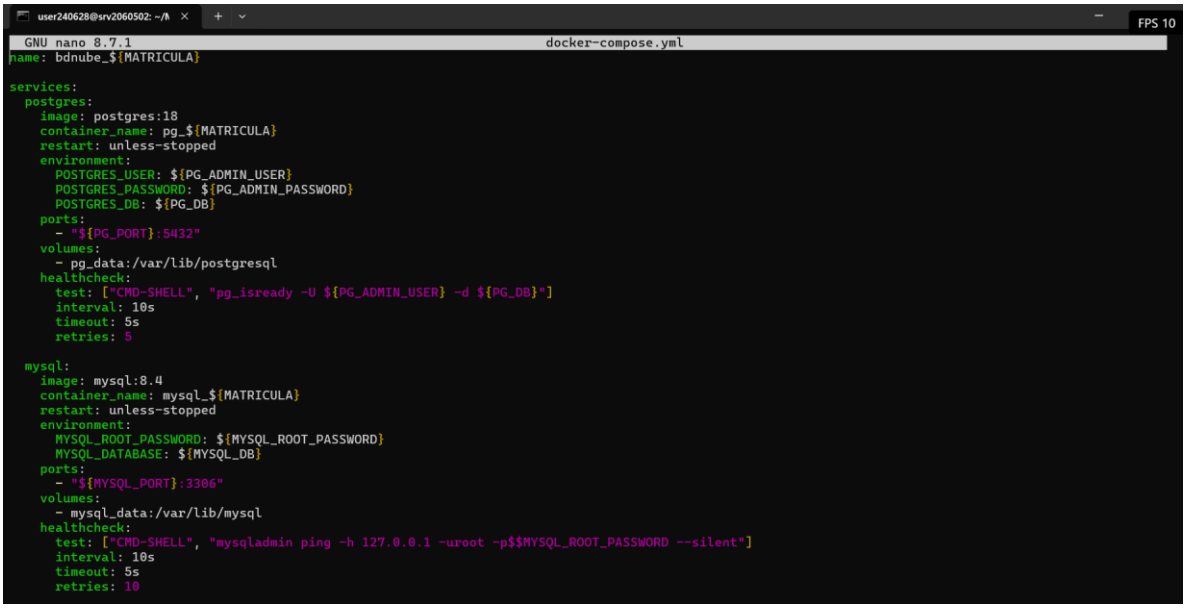
Antes del despliegue final, el VPS tenía contenedores de pruebas anteriores (pgadmin240628, pma_240628 y mysql_240628).

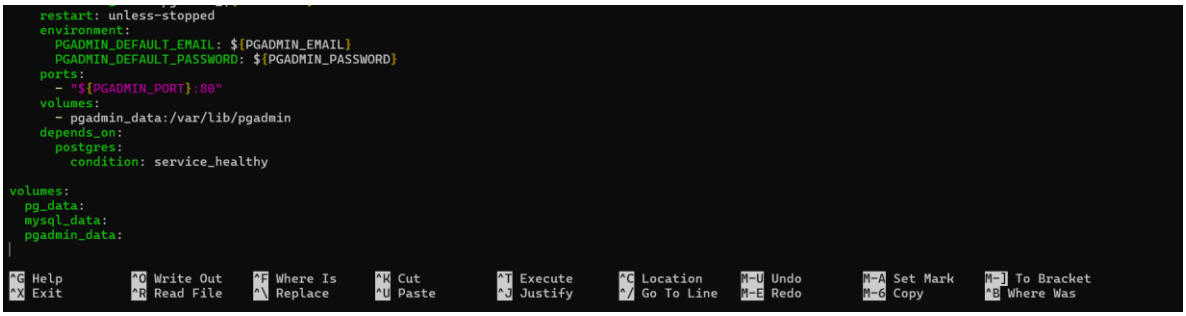
```
NAMES           IMAGE                    STATUS
pgadmin240628   dpage/pgadmin4:latest    Up 28 minutes
pma_240628      phpmyadmin:latest        Up 3 hours
mysql_240628    mysql:8.0                Up 3 hours
```

Se eliminaron por nombre, junto con su carpeta de proyecto, para empezar desde cero sin afectar a los contenedores de los demás compañeros del VPS.

El proyecto propio e independiente está en ~/Manual_VPS_BDs_Relacionales. Las credenciales están en un archivo .env con permisos 600, que no se sube a GitHub; en el repositorio solo se entrega .env.example.

docker-compose.yml:





Se usó PostgreSQL 18, la versión estable actual (la versión 19 aún no tiene lanzamiento estable). En PostgreSQL 18 el volumen se monta en /var/lib/postgresql, no en /data, y así quedó configurado.

Volúmenes persistentes. Se crearon bdnube_240628_pg_data, bdnube_240628_mysql_data y bdnube_240628_pgadmin_data, que conservan los datos al recrear los contenedores.

Resultado del despliegue (docker compose ps): los tres contenedores quedaron en ejecución, pg_240628 en estado healthy, y publicados en los puertos 40628, 40629 y 40630.

docker compose up -d

```
user240628@srv2060502:~/Manual_VPS_BDs_Relacionales$ docker compose up -d
[+] up 3/3
 ✔Container pgadmin_240628 Running
 ✔Container pg_240628      Healthy
 ✔Container mysql_240628   Running
```

docker compose ps

```
user240628@srv2060502:~/Manual_VPS_BDs_Relacionales$ docker compose ps
NAME              IMAGE                    COMMAND                  SERVICE    CREATED        STATUS                    PORTS
mysql_240628      mysql:8.4                "docker-entrypoint.s…"   mysql      13 hours ago   Up 13 hours (healthy)     33060/tcp, 0.0.0.0:40629->3306/tcp,
 [::]:40629->3306/tcp
pg_240628         postgres:18              "docker-entrypoint.s…"   postgres   13 hours ago   Up 13 hours (healthy)     0.0.0.0:40628->5432/tcp, [::]:40628
->5432/tcp
pgadmin_240628    dpage/pgadmin4:latest    "/entrypoint.sh"         pgadmin    13 hours ago   Up 13 hours               443/tcp, 0.0.0.0:40630->80/tcp, [::
]:40630->80/tcp
```

docker compose logs --tail 50

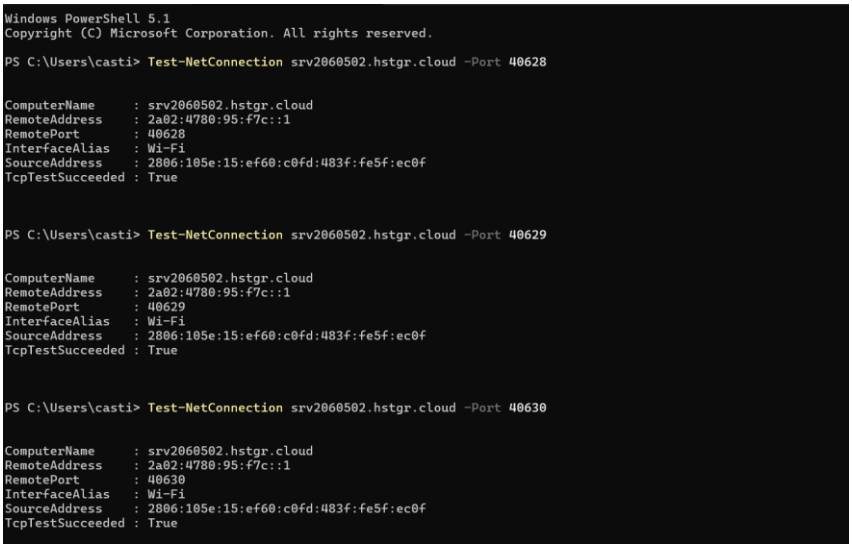
```
user240628@srv2060502:~/Manual_VPS_BDs_Relacionales$ docker compose up -d
docker compose ps
docker compose logs --tail 50
[+] up 3/3
 ✔Container mysql_240628   Running                                                                                                0.0s
 ✔Container pgadmin_240628 Running                                                                                                0.0s
 ✔Container pg_240628      Healthy                                                                                                0.5s
NAME              IMAGE                    COMMAND                  SERVICE    CREATED        STATUS                    PORTS
mysql_240628      mysql:8.4                "docker-entrypoint.s…"   mysql      13 hours ago   Up 13 hours (healthy)     33060/tcp, 0.0.0.0:40629->3306/tcp,
 [::]:40629->3306/tcp
pg_240628         postgres:18              "docker-entrypoint.s…"   postgres   13 hours ago   Up 13 hours (healthy)     0.0.0.0:40628->5432/tcp, [::]:40628
->5432/tcp
pgadmin_240628    dpage/pgadmin4:latest    "/entrypoint.sh"         pgadmin    13 hours ago   Up 13 hours               443/tcp, 0.0.0.0:40630->80/tcp, [::
]:40630->80/tcp
mysql_240628      | 2026-10-25 03:17:54+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.13-1.el9 started.
mysql_240628      | 2026-10-25 03:17:55+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
mysql_240628      | 2026-10-25 03:17:55+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.13-1.el9 started.
mysql_240628      | 2026-10-25 03:17:55+00:00 [Note] [Entrypoint]: Initializing database files
mysql_240628      | 2026-10-25T03:17:55.372000Z 0 [System] [MY-015017] [Server] MySQL Server Initialization - start.
mysql_240628      | 2026-10-25T03:17:55.374439Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.4.13) initializing of server in progress as pr
ocess 79
mysql_240628      | 2026-10-25T03:17:55.374528Z 0 [System] [MY-015150] [Server] Server version with update: '8.4.13.0'
mysql_240628      | 2026-10-25T03:17:55.391216Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
mysql_240628      | 2026-10-25T03:17:56.790814Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
mysql_240628      | 2026-10-25T03:17:58.103874Z 6 [Warning] [MY-010453] [Server] root@localhost is created with an empty password ! Please consider switc
hing off the --initialize-insecure option.
mysql_240628      | 2026-10-25T03:18:00.613995Z 0 [System] [MY-015018] [Server] MySQL Server Initialization - end.
mysql_240628      | 2026-10-25 03:18:00+00:00 [Note] [Entrypoint]: Database files initialized
mysql_240628      | 2026-10-25 03:18:00+00:00 [Note] [Entrypoint]: Starting temporary server
mysql_240628      | 2026-10-25T03:18:00.806576Z 0 [System] [MY-015015] [Server] MySQL Server - start.
mysql_240628      | 2026-10-25T03:18:02.047904Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.4.13) starting as process 118
mysql_240628      | 2026-10-25T03:18:02.047909Z 0 [System] [MY-015150] [Server] Server version with update: '8.4.13.0'
mysql_240628      | 2026-10-25T03:18:02.096197Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
mysql_240628      | 2026-10-25T03:18:03.515268Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
mysql_240628      | 2026-10-25T03:18:03.974134Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
mysql_240628      | 2026-10-25T03:18:03.974166Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are n
ow supported for this channel.
mysql_240628      | 2026-10-25T03:18:03.976520Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the
 path is accessible to all OS users. Consider choosing a different directory.
mysql_240628      | 2026-10-25T03:18:03.996719Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Socket: /var/run/mysqld/mysqlx.sock
mysql_240628      | 2026-10-25T03:18:03.997382Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.4.13'  socket: '/var
/run/mysqld/mysqld.sock'  port: 0  MySQL Community Server - GPL.
mysql_240628      | 2026-10-25 03:18:04+00:00 [Note] [Entrypoint]: Temporary server started.
```



Puertos y conexiones remotas

Según la lista de verificación, PostgreSQL usa los cinco dígitos de la matrícula (40628) y MySQL ese número más uno (40629); ambos son válidos porque están dentro del rango 1 a 65535. Para pgAdmin Web se eligió el 40630. Antes del despliegue final se comprobó con ss -tlnp que los puertos estuvieran libres tras eliminar los contenedores anteriores.

PostgreSQL y MySQL aceptan conexiones remotas porque sus puertos están publicados en todas las interfaces del VPS (0.0.0.0), y la autenticación se hace con las cuentas administradoras definidas en el archivo .env.

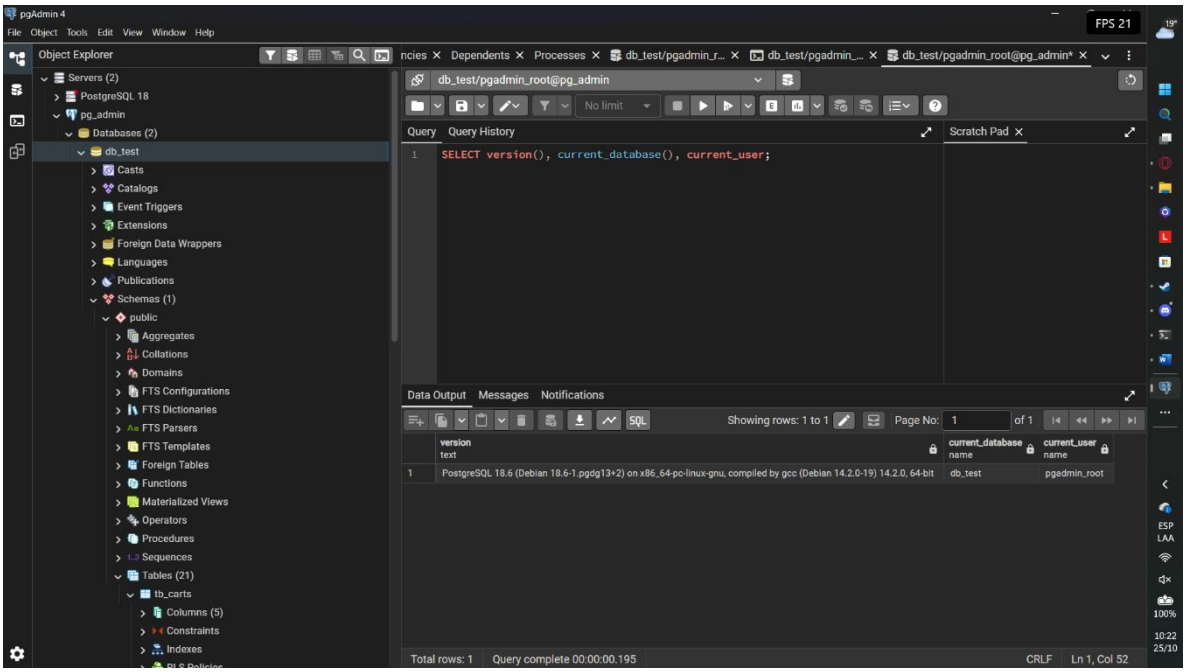


Conexión remota a PostgreSQL. La conexión se hizo desde mi computadora, fuera de los contenedores y sin usar SSH, con pgAdmin 4 de escritorio y la cuenta administradora del motor. A diferencia de pgAdmin Web, que se conecta por la red interna de Docker (host postgres, puerto 5432), el cliente externo usa el dominio del VPS y el puerto publicado.

Parámetro	Valor
Host	srv2060502.hstgr.cloud
Puerto	40628
Base de datos	db_test
Usuario	pgadmin_root (cuenta administradora)
Modo SSL	prefer

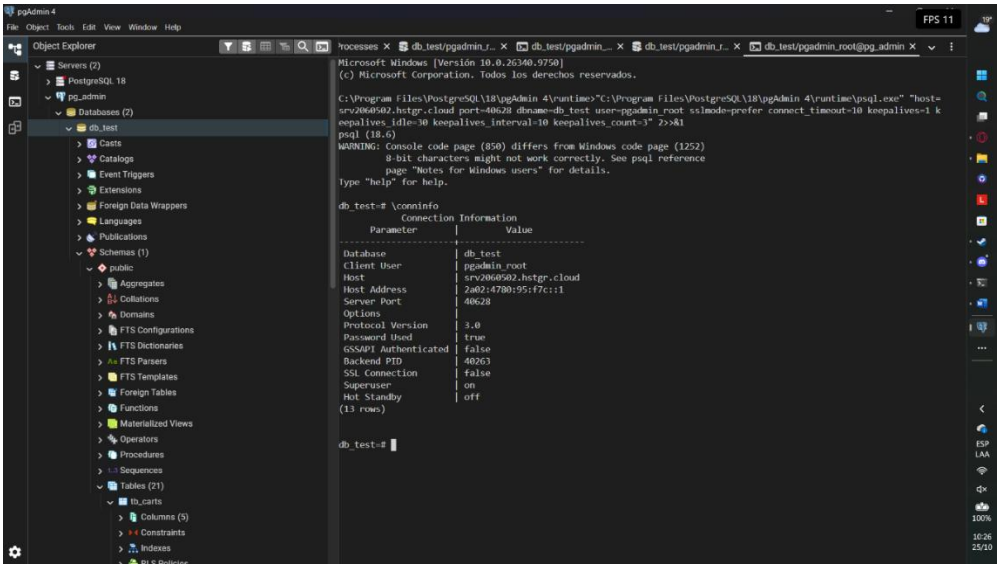
La conexión se comprobó con una consulta SQL en el Query Tool y con el comando \conninfo en la consola PSQL Tool de pgAdmin:

SELECT version(), current_database(), current_user;



La consulta devuelve PostgreSQL 18, la base db_test y el usuario pgadmin_root, y el árbol de objetos muestra las 21 tablas del esquema public.

\conninfo



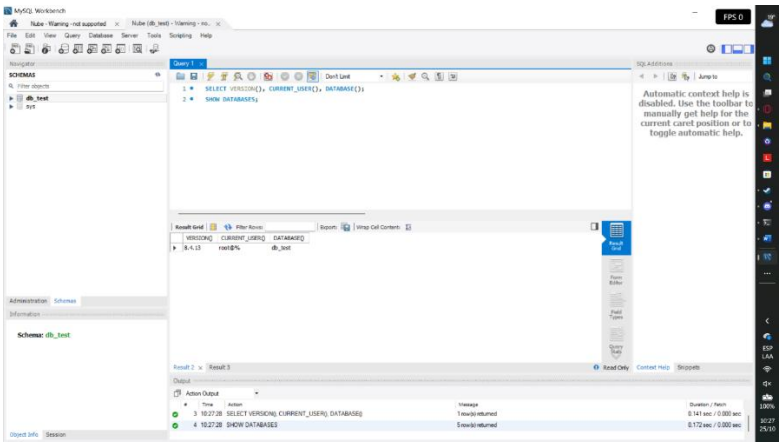
Conexión remota a MySQL. Se hizo desde mi computadora, fuera de los contenedores, con MySQL Workbench y la cuenta administradora root, usando el dominio del VPS y el puerto publicado.

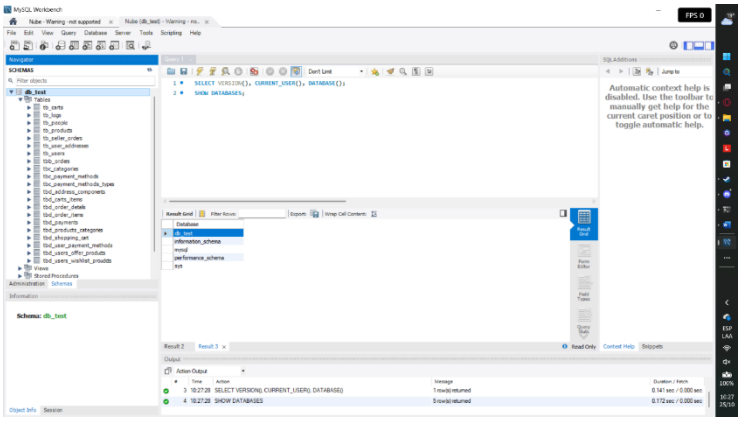
Parámetro	Valor
Hostname	srv2060502.hstgr.cloud
Port	40629
Username	root (cuenta administradora)

La conexión se comprobó con estas consultas:

SELECT VERSION(), CURRENT_USER(), DATABASE();

SHOW DATABASES;





La primera devuelve la versión 8.4 de MySQL y la cuenta con la que se entró, y SHOW DATABASES muestra la base db_test.

Roles, usuarios y privilegios

Roles obligatorios: superadmin, admin, seller, buyer, support, common y not_registered. Cuentas personales solicitadas en cada motor: un superadmin, un admin, dos seller y un support.

Se crearon los siete roles del contexto e-commerce y las cuentas personales en los dos motores, con las mismas reglas de negocio de la Práctica 02. En MySQL se usó el script remote_users_privileges_creation.sql y en PostgreSQL el script 04_roles_privilegios.sql, que replica el anterior con la sintaxis de PostgreSQL. Los roles se llaman superadmin, admin, seller, buyer, support, common y user_not_registered.

Privilegios por rol (iguales en MySQL y PostgreSQL):

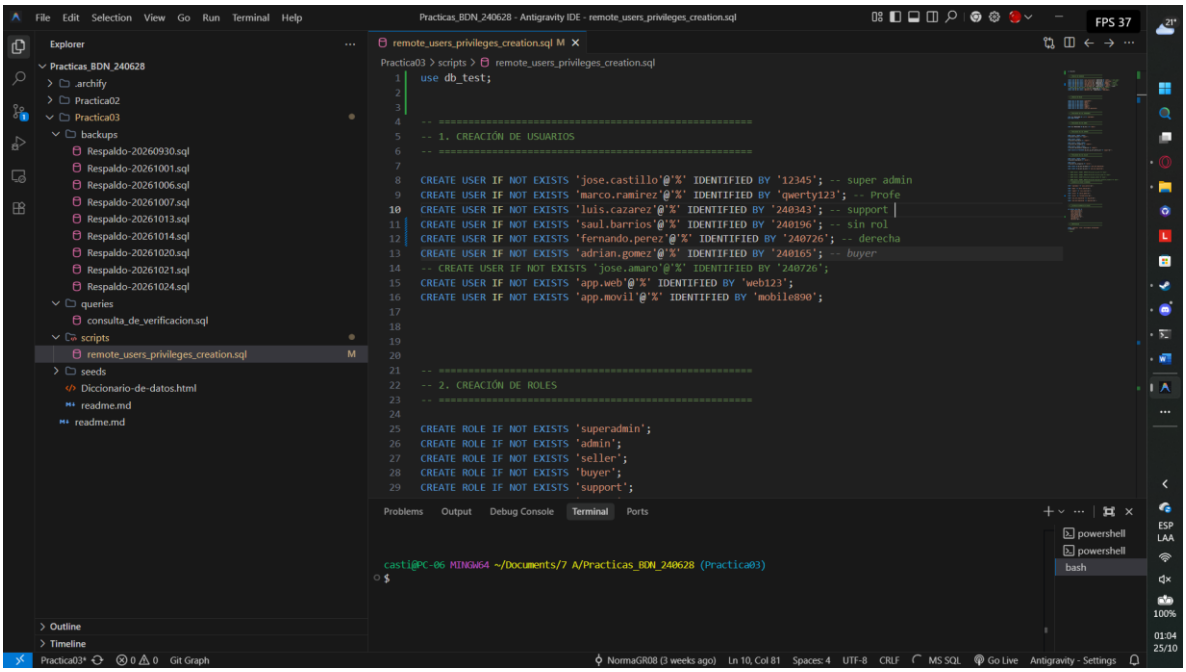
Rol	Privilegios sobre db_test

superadmin	Todos los privilegios sobre todo el servidor, con la capacidad de otorgarlos
admin	Todos los privilegios sobre la base db_test
support	tb_users, tbc_categories y tbd_products_categories: SELECT, INSERT y UPDATE. tb_products: SELECT y UPDATE. Ejecutar sp_soft_delete_user
seller	tb_products: SELECT, INSERT y UPDATE. tbc_categories: SELECT
user_not_registered	INSERT en tb_people y tb_users
buyer y common	Creados sin privilegios

Cuentas personales y roles asignados:

Cuenta	Rol
jose.castillo	superadmin
marco.ramirez	admin
luis.cazarez	support
adrian.gomez	seller
fernando.perez	seller
saul.barrios	Sin rol asignado
app.web y app.movil	user_not_registered

Cumplen las cinco cuentas que pide la práctica: un superadmin, un admin, dos seller y un support.

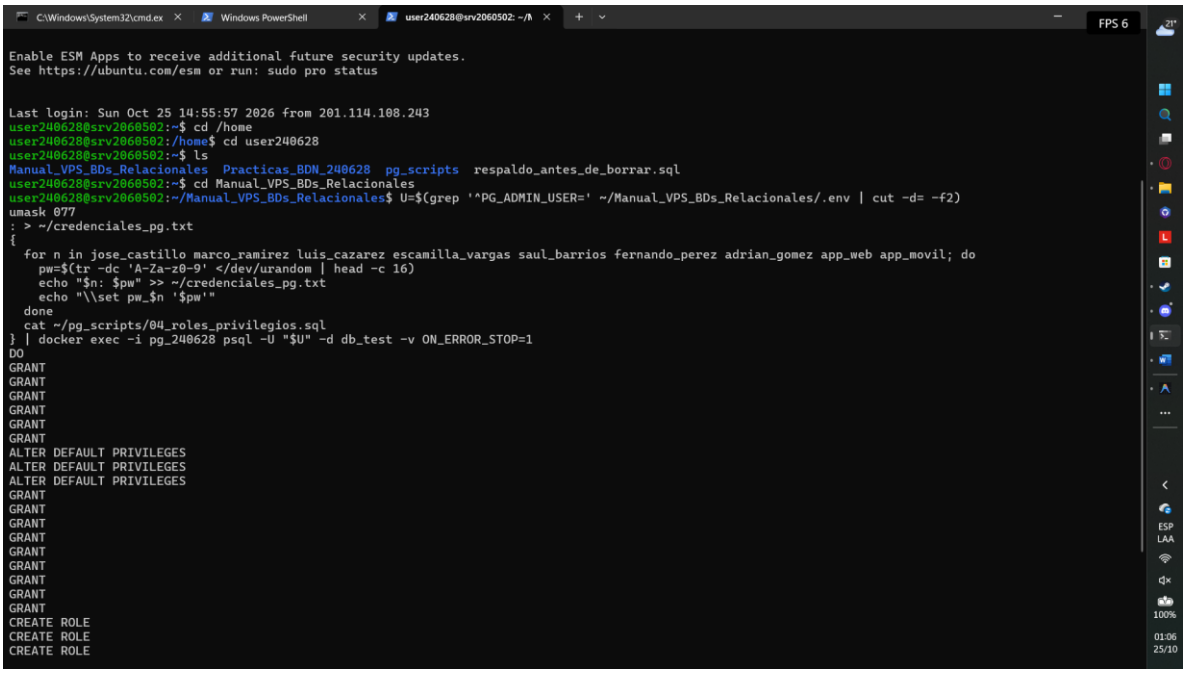


PostgreSQL. Los roles se crean como roles de grupo sin inicio de sesión, y cada cuenta se crea con su contraseña recibida como variable de psql, sin escribirla en el archivo:

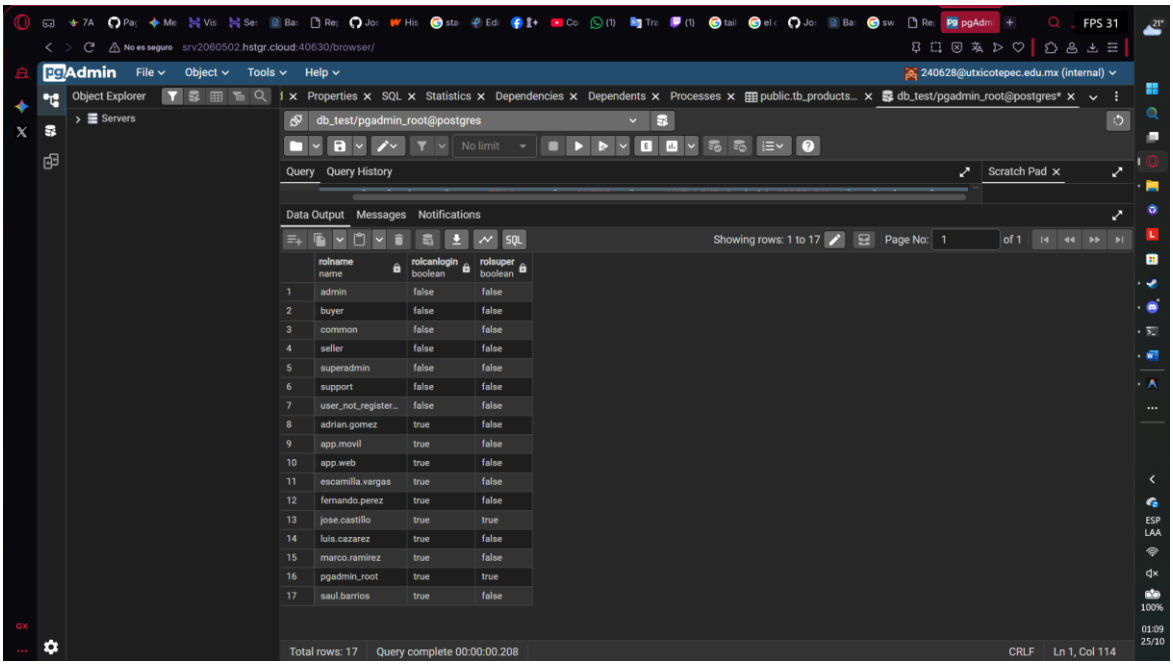
Ejemplo:

```
CREATE ROLE support NOLOGIN;
GRANT SELECT, INSERT, UPDATE ON tb_users TO support;
GRANT EXECUTE ON FUNCTION sp_soft_delete_user(integer) TO support;
CREATE ROLE "luis.cazarez" LOGIN PASSWORD '<contraseña>' IN ROLE support;
```

El script se ejecutó dentro del contenedor de PostgreSQL, y las contraseñas de las cuentas se generaron al azar y se guardaron fuera del repositorio:

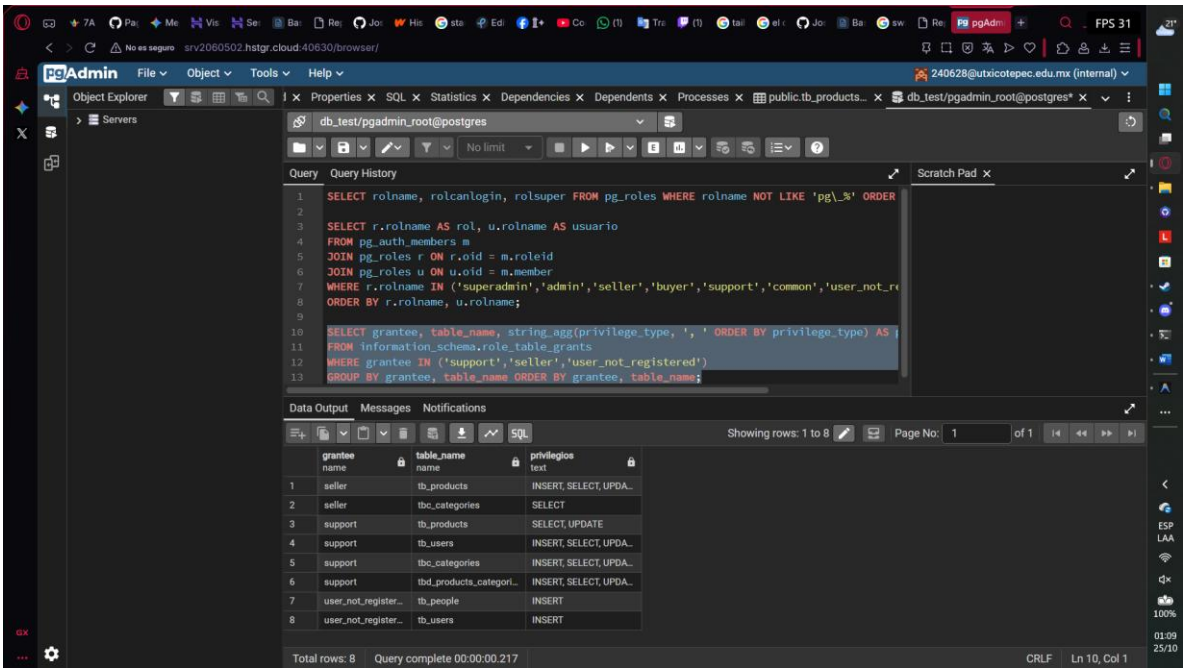


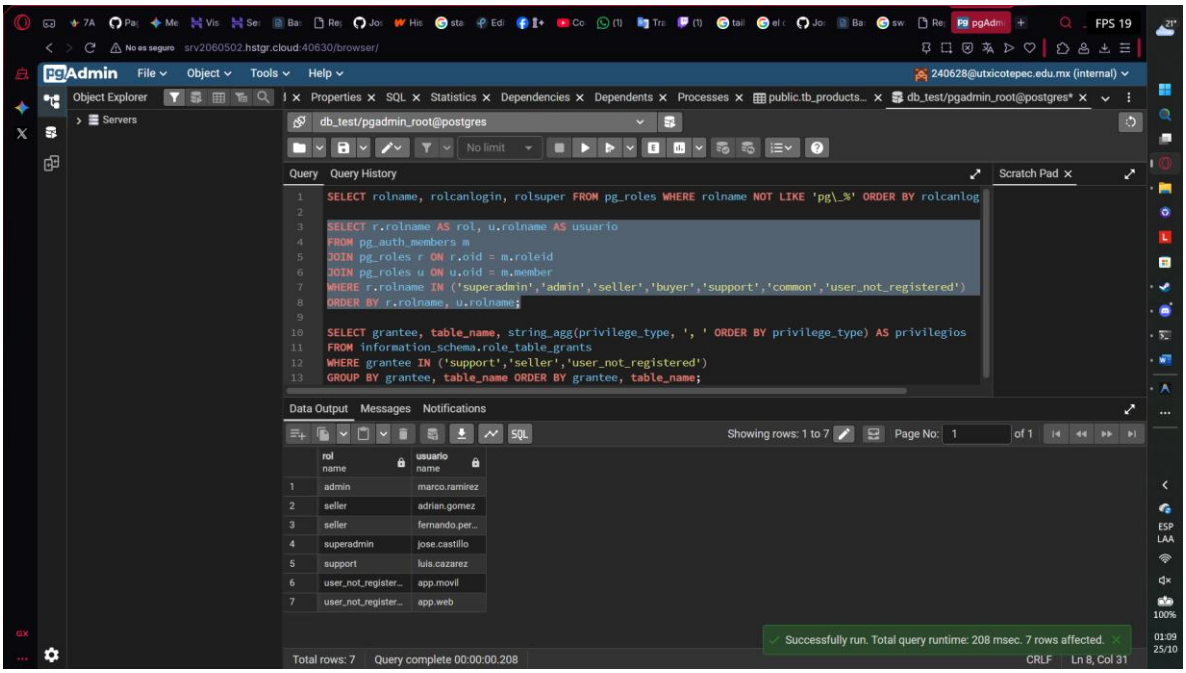
Prueba funcional: con SET ROLE "adrian.gomez"; una consulta a tb_products funciona y una a tb_users responde permission denied, porque un seller no puede leer usuarios. Se vuelve al usuario administrador con RESET ROLE;.



Diferencias entre los dos motores. En MySQL el rol superadmin recibe ALL PRIVILEGES ON *.*; en PostgreSQL el atributo SUPERUSER no se hereda por pertenecer a un rol, por lo que se asigna directamente a la cuenta jose.castillo. El rol

admin de PostgreSQL recibe todos los privilegios sobre tablas, secuencias y rutinas, pero modificar o borrar tablas existentes exige ser su dueño, algo que no se concede con GRANT. Los nombres de cuenta con punto, como jose.castillo, van entre comillas dobles en PostgreSQL.





Firewall y restricciones de acceso. Se permitieron en el firewall del sistema (ufw) los tres puertos del proyecto, desde la sesión de administrador (root), y se comprobó el resultado:

```
ufw allow 40628/tcp comment 'PostgreSQL'
ufw allow 40629/tcp comment 'MySQL'
ufw allow 40630/tcp comment 'pgAdmin Web'
ufw status verbose
ss -tlnp | grep -E '40628|40629|40630'
```



Descripción de la captura. Los tres comandos ufw allow respondieron Skipping adding existing rule, tanto para IPv4 como para IPv6 (v6): las reglas de los puertos 40628, 40629 y 40630 ya estaban registradas. Sin embargo, ufw status verbose muestra Status: inactive, lo que significa que ufw tiene las reglas definidas, pero no está activo, por lo que por sí solo no filtra el tráfico. La salida de ss -tlnp muestra que los tres puertos están en escucha (LISTEN) en todas las interfaces, 0.0.0.0 para IPv4 y [::] para IPv6, atendidos por el proceso docker-proxy, que es el que Docker usa para publicar los puertos de los contenedores: el 40628 corresponde a PostgreSQL, el 40629 a MySQL y el 40630 a pgAdmin Web.

Bases de datos y objetos relacionales

Se trabajó con la base de datos db_test, que contiene 21 tablas (incluyendo tb_users y tb_products), 19 vistas de trazabilidad (vw_trazabilidad_*), 6 procedimientos almacenados y triggers de auditoría sobre las operaciones de inserción, actualización y eliminación, que registran los eventos en la tabla tb_logs.

MySQL

La base se restauró en el contenedor mysql_240628 desde el respaldo Respaldo-20261024.sql, creado en MySQL 8.0:

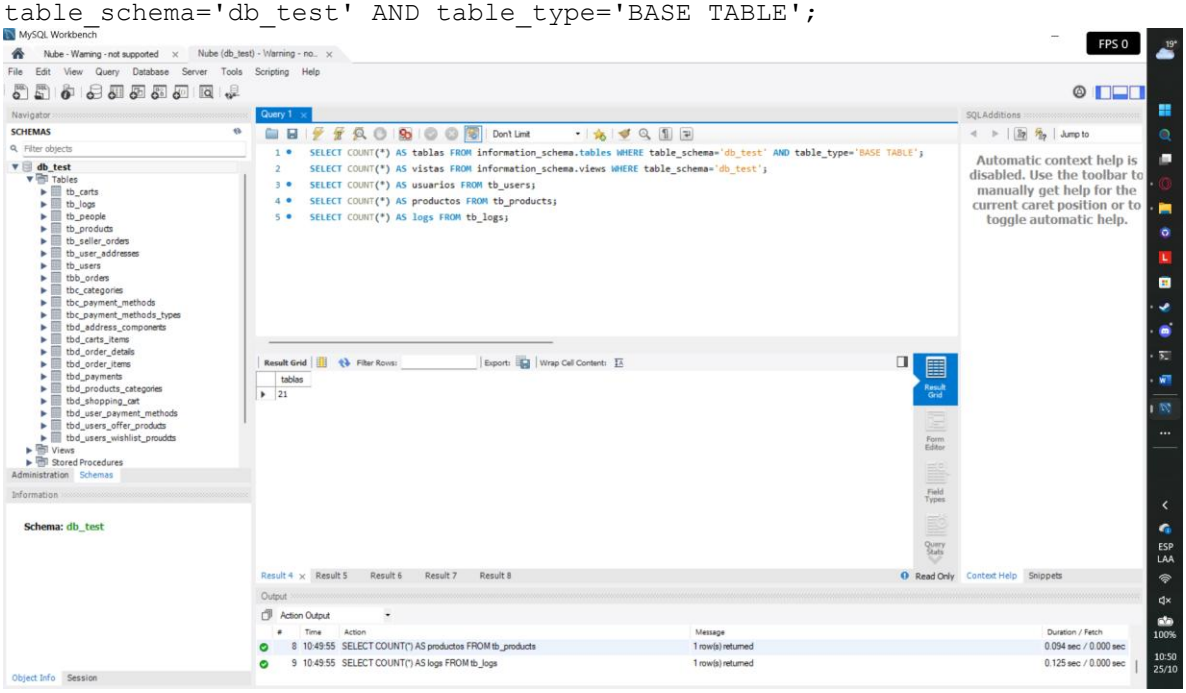
```
read -s -p "Clave root de MySQL: " P; echo
( echo "SET SESSION restrict_fk_on_non_standard_key=OFF;"; sed 's/\r$//'
Respaldo-20261024.sql ) | docker exec -i -e MYSQL_PWD="$P" mysql_240628
mysql -uroot
unset P
```

La restauración falló primero con el error 6125: Failed to add the foreign key constraint en la tabla tbd_order_items. La causa es que MySQL 8.4 prohíbe, por defecto, llaves foráneas que apuntan a columnas no únicas

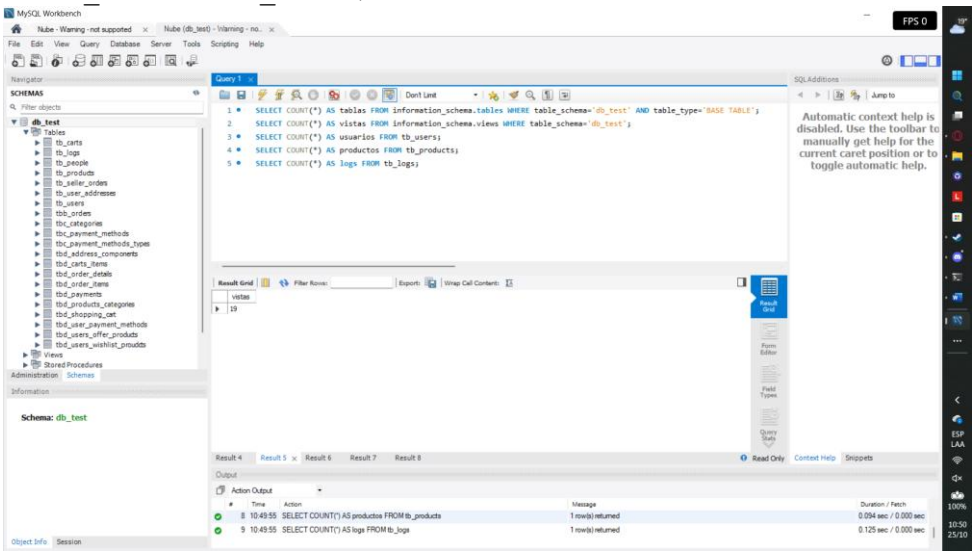
(tbd_users_offer_products), algo que MySQL 8.0 sí permitía. La solución fue desactivar restrict_fk_on_non_standard_key únicamente durante la sesión de restauración.

Verificación de objetos:

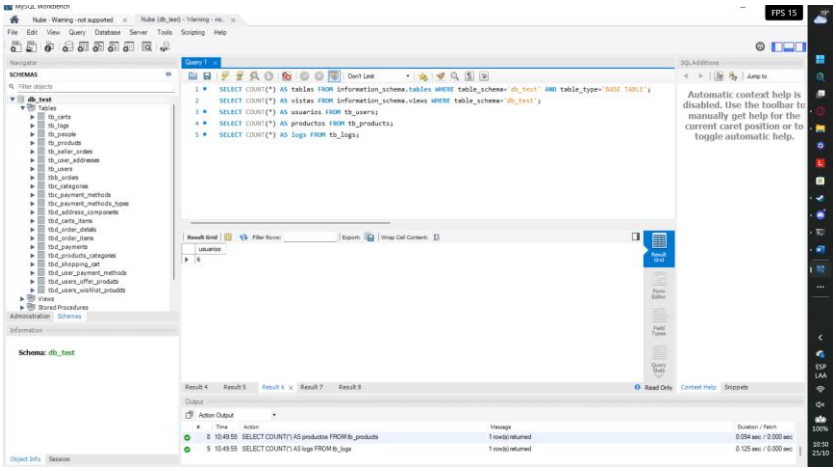
```
SELECT COUNT(*) AS tablas FROM information_schema.tables WHERE
table_schema='db_test' AND table_type='BASE TABLE';
```



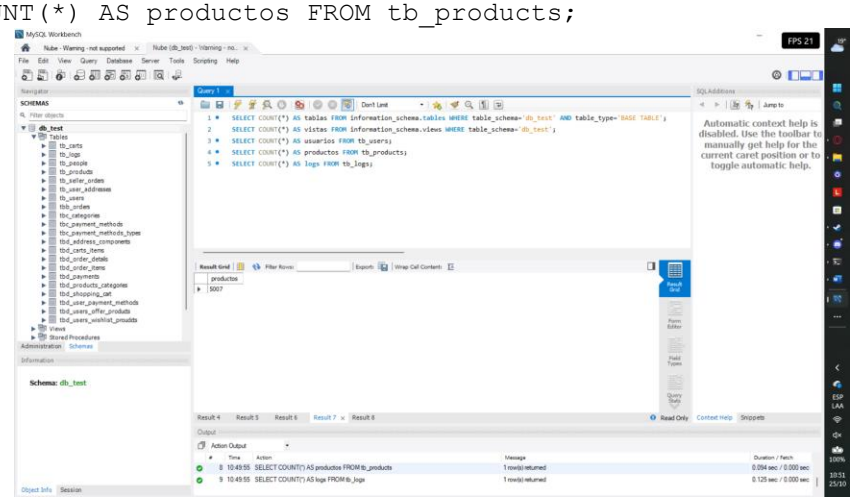

```
SELECT COUNT(*) AS vistas FROM information_schema.views WHERE
table_schema='db_test';
```



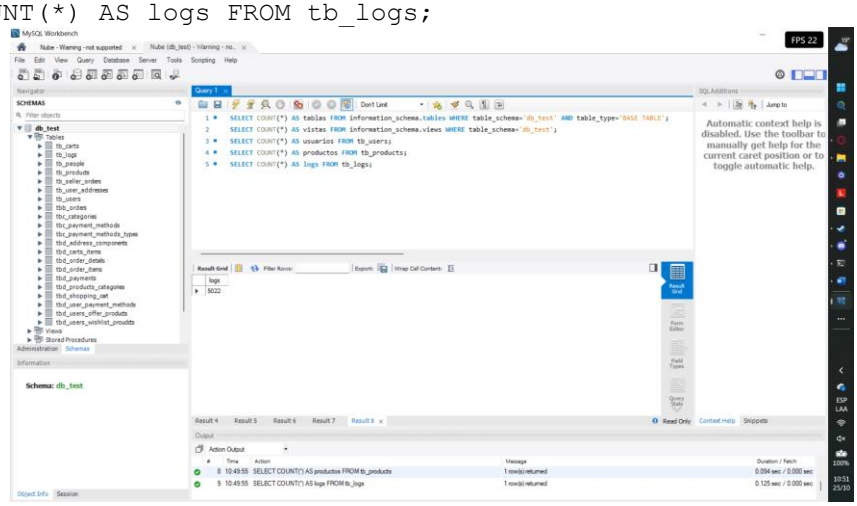

```
SELECT COUNT(*) AS usuarios FROM tb_users;
```



```
SELECT COUNT(*) AS productos FROM tb_products;
```



```
SELECT COUNT(*) AS logs FROM tb_logs;
```



PostgreSQL

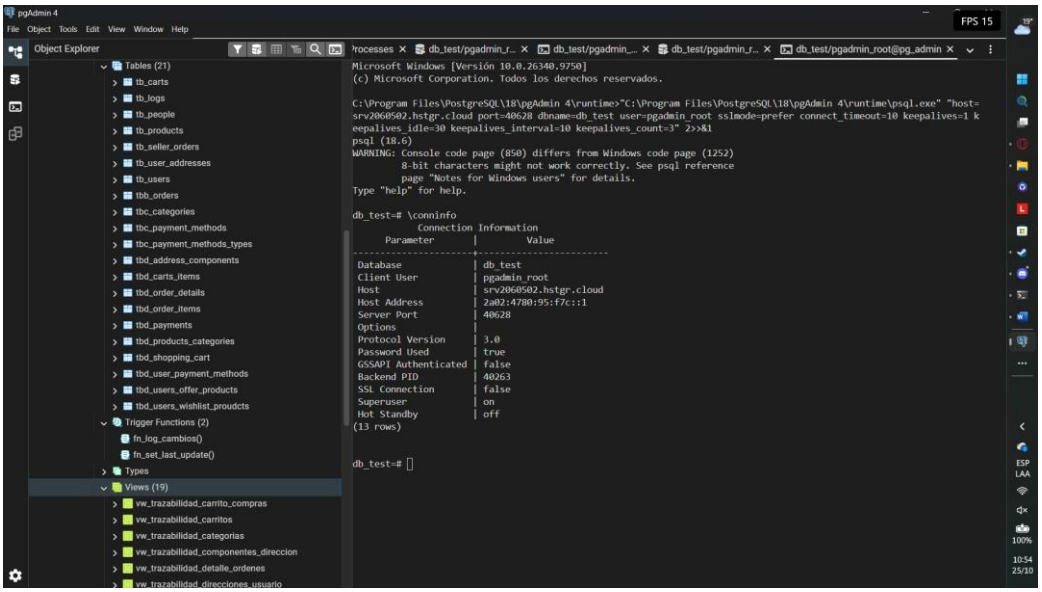
Como un respaldo de MySQL no se puede cargar directamente en PostgreSQL, la base se convirtió a tres scripts, que se cargaron en este orden dentro del contenedor pg_240628:

1. 01_esquema.sql: tipos ENUM, 21 tablas, índices y llaves foráneas.
2. 02_datos.sql: datos de tb_users, tb_products y tb_logs.
3. 03_logica.sql: funciones, triggers, procedimientos y vistas.

El orden es importante: los triggers se crean al final para que no dupliquen los registros de tb_logs al cargar los datos.

```
U=$(grep '^PG_ADMIN_USER=' .env | cut -d= -f2)
for f in 01_esquema 02_datos 03_logica; do
  docker exec -i pg_240628 psql -U "$U" -d db_test -v ON_ERROR_STOP=1 -1
-f - < ~/pg_scripts/$f.sql || break
done
```

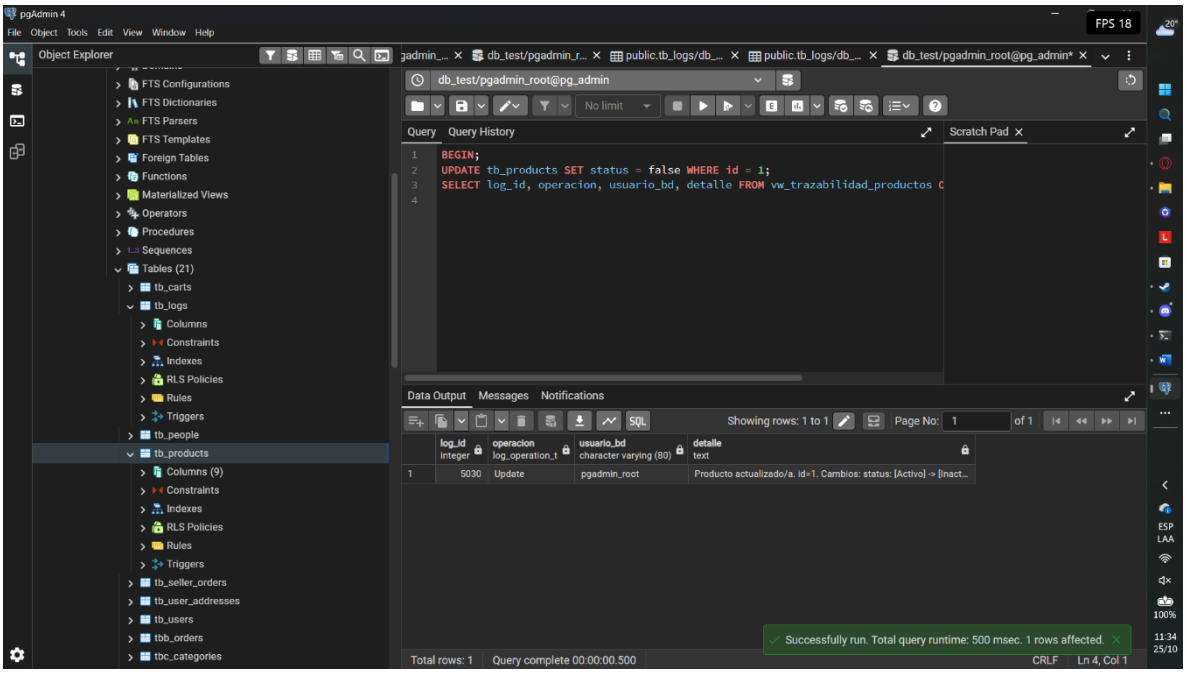
MySQL	PostgreSQL
AUTO_INCREMENT	GENERATED BY DEFAULT AS IDENTITY
bit(1)	boolean
datetime	timestamp
ON UPDATE CURRENT_TIMESTAMP	trigger fn_set_last_update
triggers con el cuerpo dentro	función genérica fn_log_cambios y un CREATE TRIGGER por evento
USER()	session_user
llaves foráneas a columnas no únicas	llaves a tb_products(id) y tb_users(id)



Pruebas y resultados

Prueba del trigger y la vista de trazabilidad en PostgreSQL

Se actualizó el producto 1 dentro de una transacción que se deshizo con ROLLBACK, y se consultó la vista de trazabilidad de productos:



Resultado: la vista mostró un registro nuevo con log_id 5030, operación Update, usuario pgadmin_root y el detalle "Producto actualizado/a. id=1. Cambios: status:

[Activo] -> [Inactivo]". Esto confirma que el trigger registra la operación y que la vista la muestra. El ROLLBACK dejó el producto sin cambios.

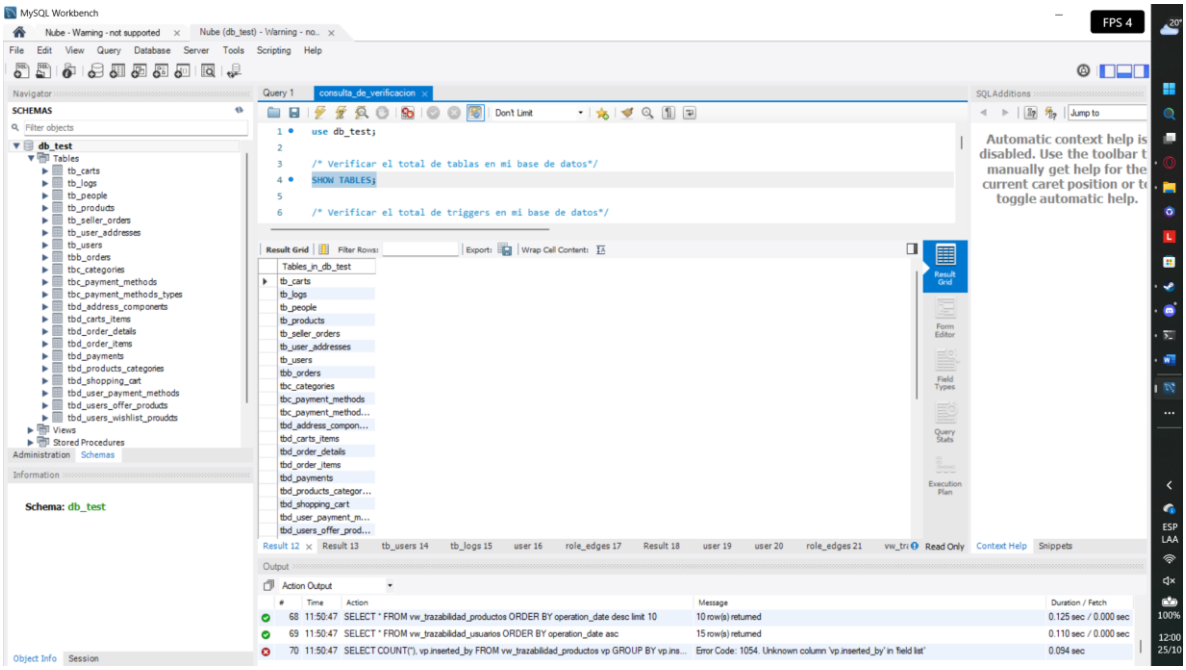
Casos de prueba de clase (tb_users y tb_products)

Los diez casos de prueba de clase son consultas SQL en MySQL que verifican las tablas, los triggers, la bitácora, las cuentas remotas, los roles y las vistas de trazabilidad. Para cada caso se muestra la operación, el resultado esperado según la base db_test restaurada y su captura.

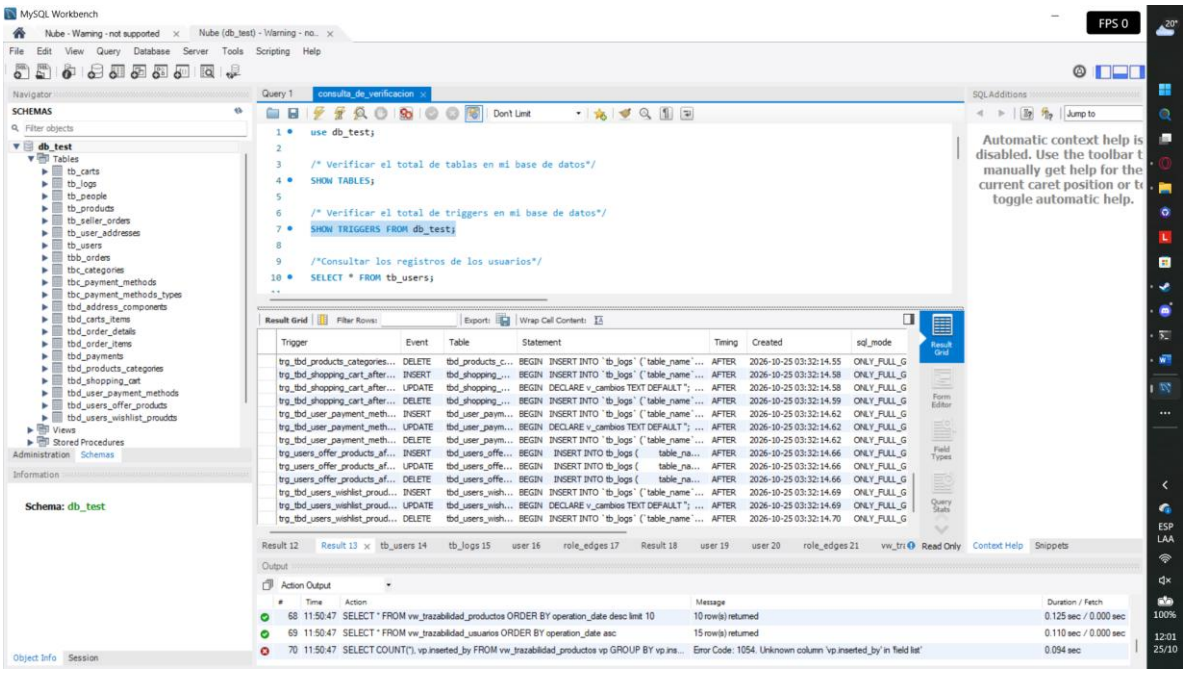
Caso	Qué verifica	Resultado esperado
01	Total de tablas de la base	40 objetos: 21 tablas y 19 vistas
02	Total de triggers de la base	57 triggers, tres por cada una de las 19 tablas auditadas
03	Registros de los usuarios (tb_users)	6 usuarios
04	Registros de la bitácora (tb_logs)	5,022 registros: 5,007 altas de productos, 9 altas de usuarios, 3 actualizaciones y 3 eliminaciones
05	Usuarios remotos creados en el servidor	Cuentas con host %: las cuentas personales del punto 4.5 (hasta crearlas, solo root)
06	Roles asignados a los usuarios remotos	Relaciones de mysql.role_edges.
07	Usuarios insertados, quién los insertó y con qué roles	3 filas: Ing-Jose-Maria, carlos_hernandez y ricardo_mendoza; la columna de roles se llena al crear los roles
08	Cuentas remotas bloqueadas	Ninguna
09	Cuentas remotas activas	root y las cuentas personales
10	Usuarios y sus roles, ordenados	Relaciones de mysql.role_edges

Consultas:

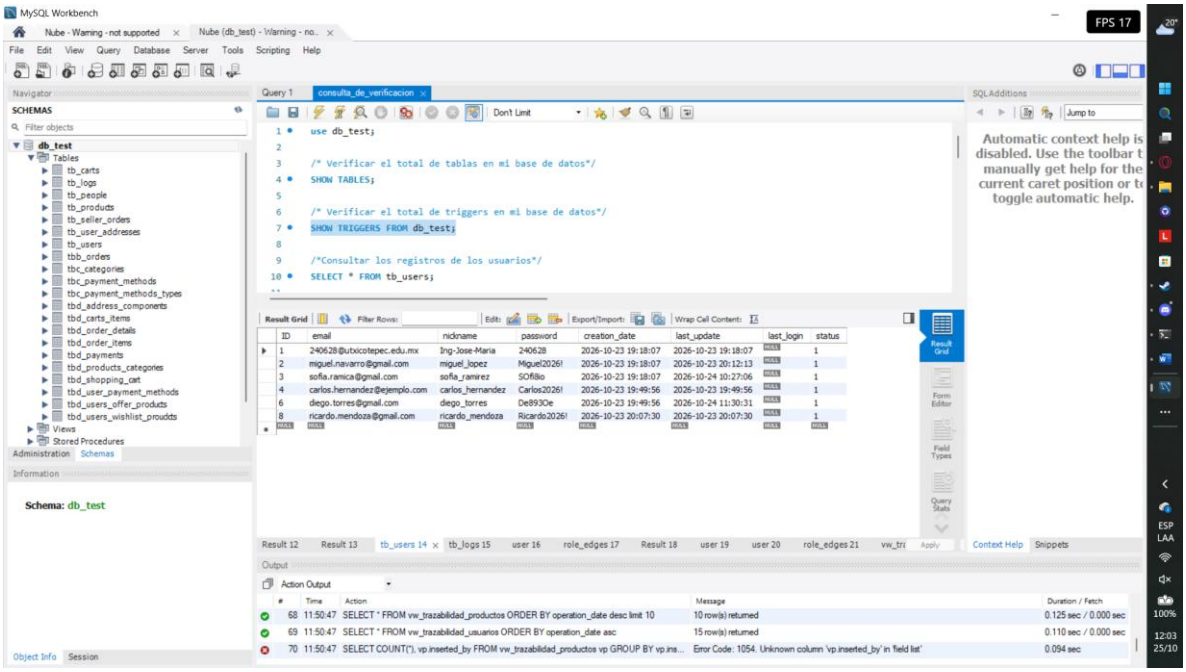
SHOW TABLES;



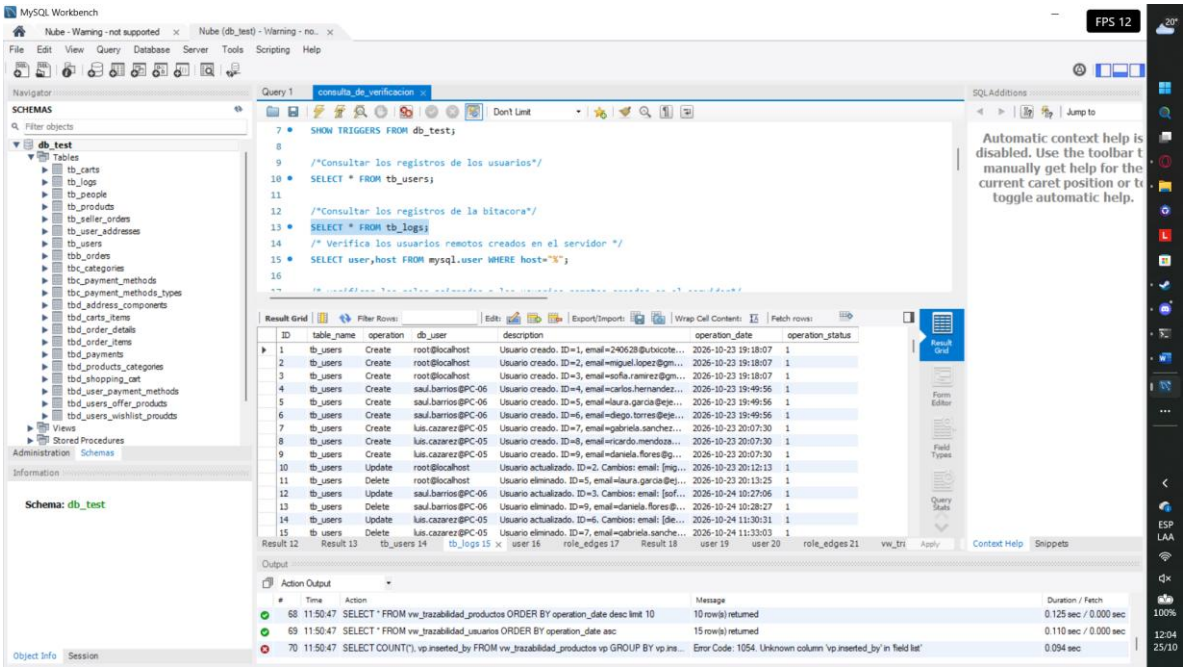
SHOW TRIGGERS FROM db_test;



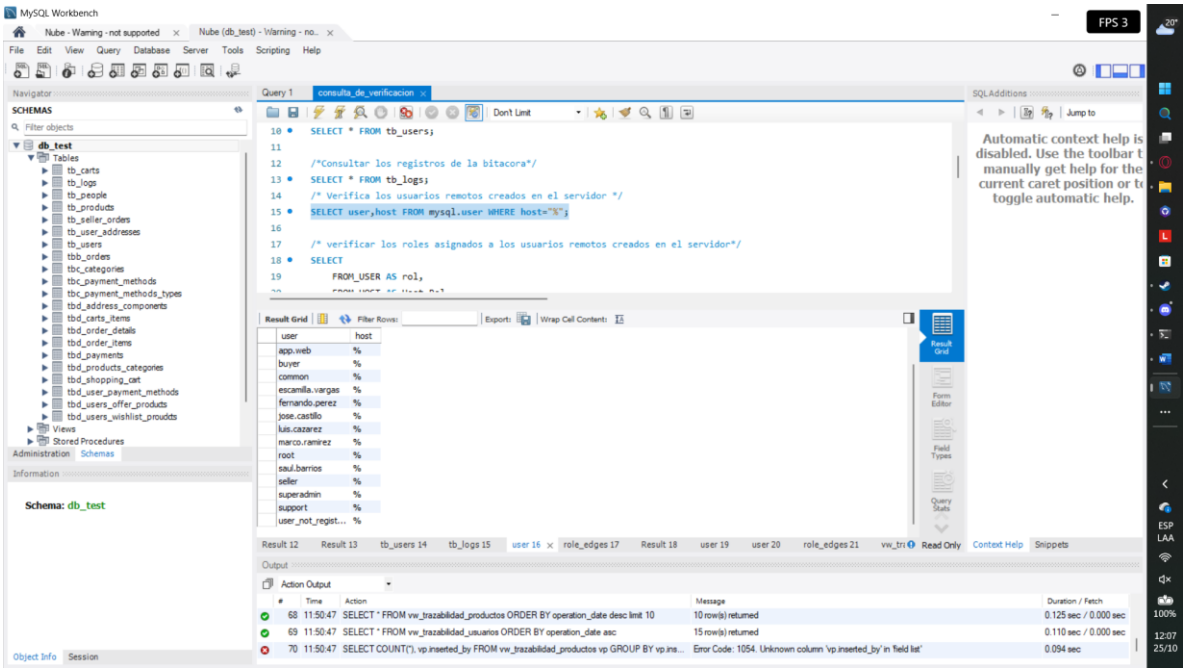
SELECT * FROM tb_users;



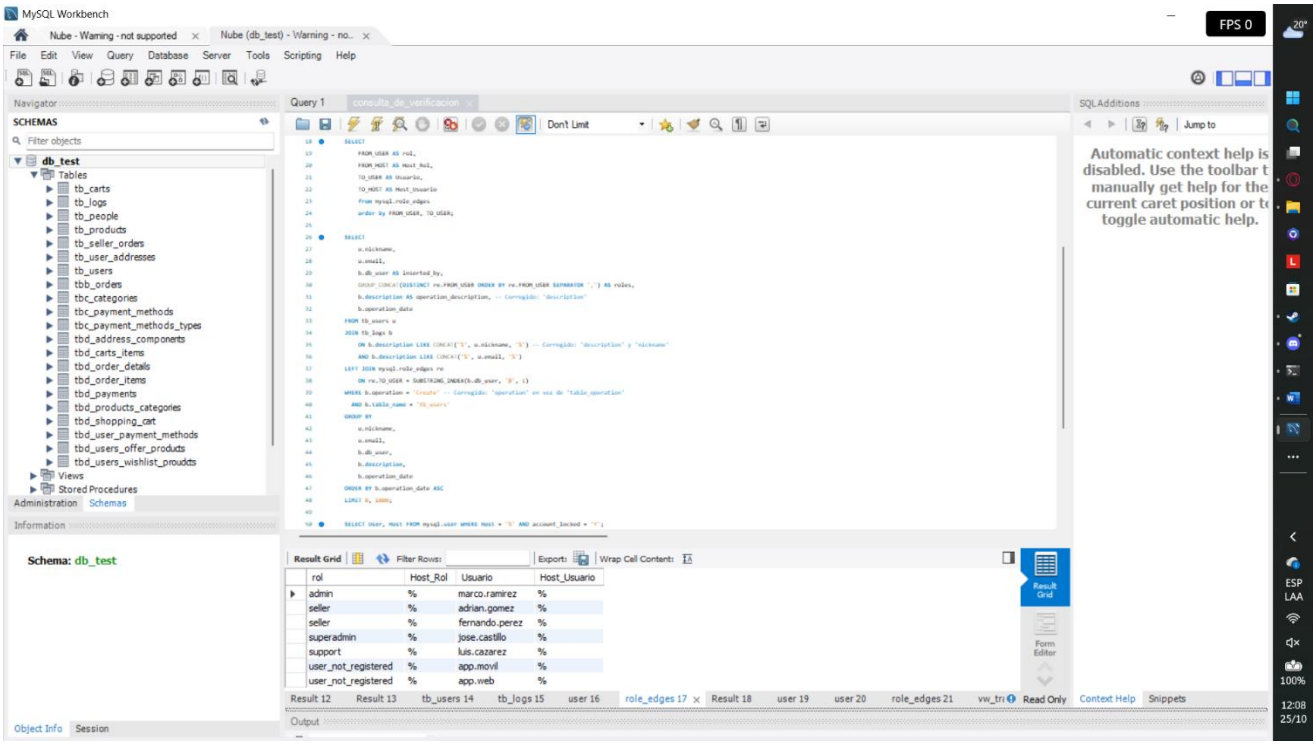
SELECT * FROM tb_logs;



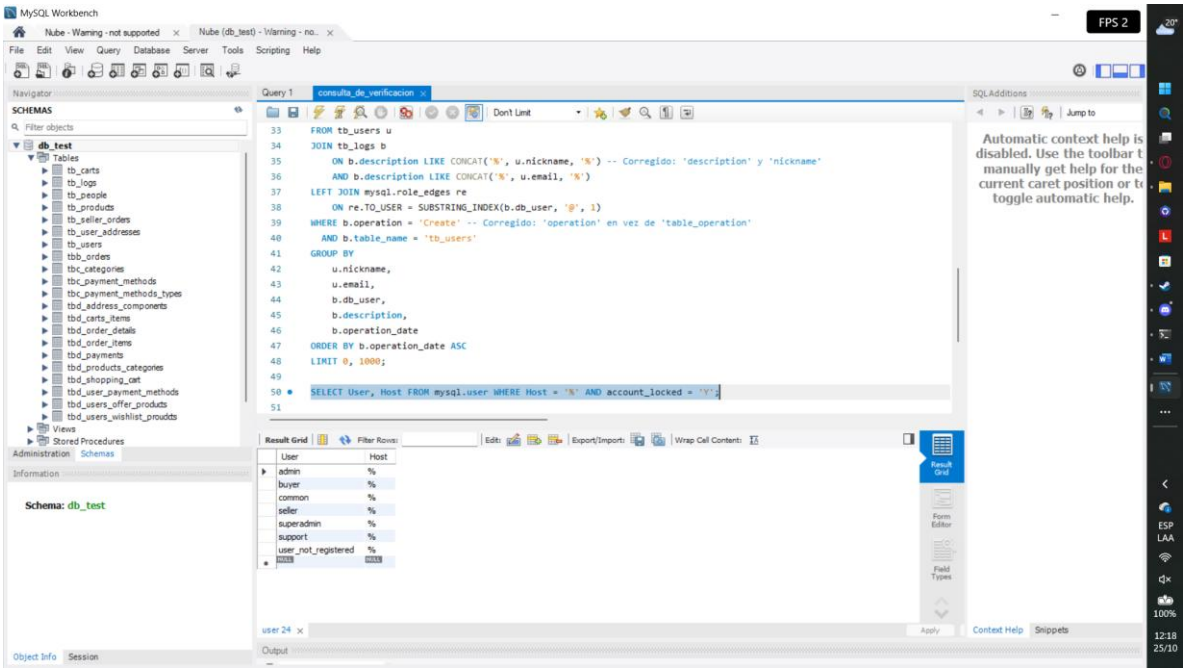
SELECT user,host FROM mysql.user WHERE host="%";



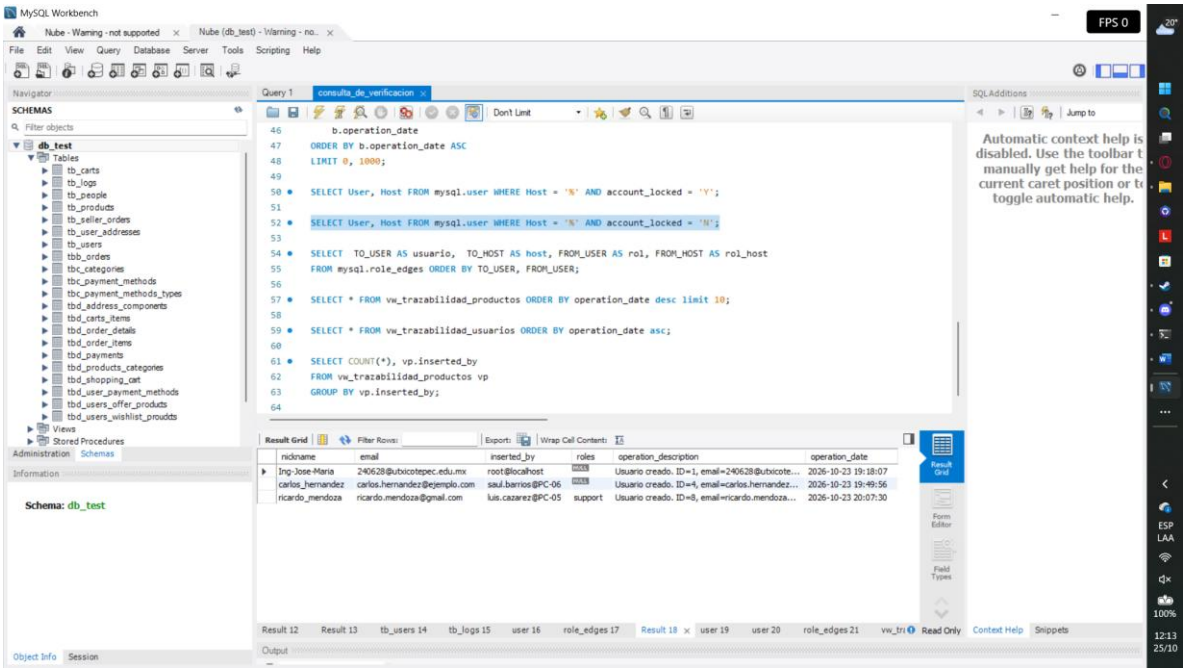
Usuarios insertados, quién los insertó y con qué roles



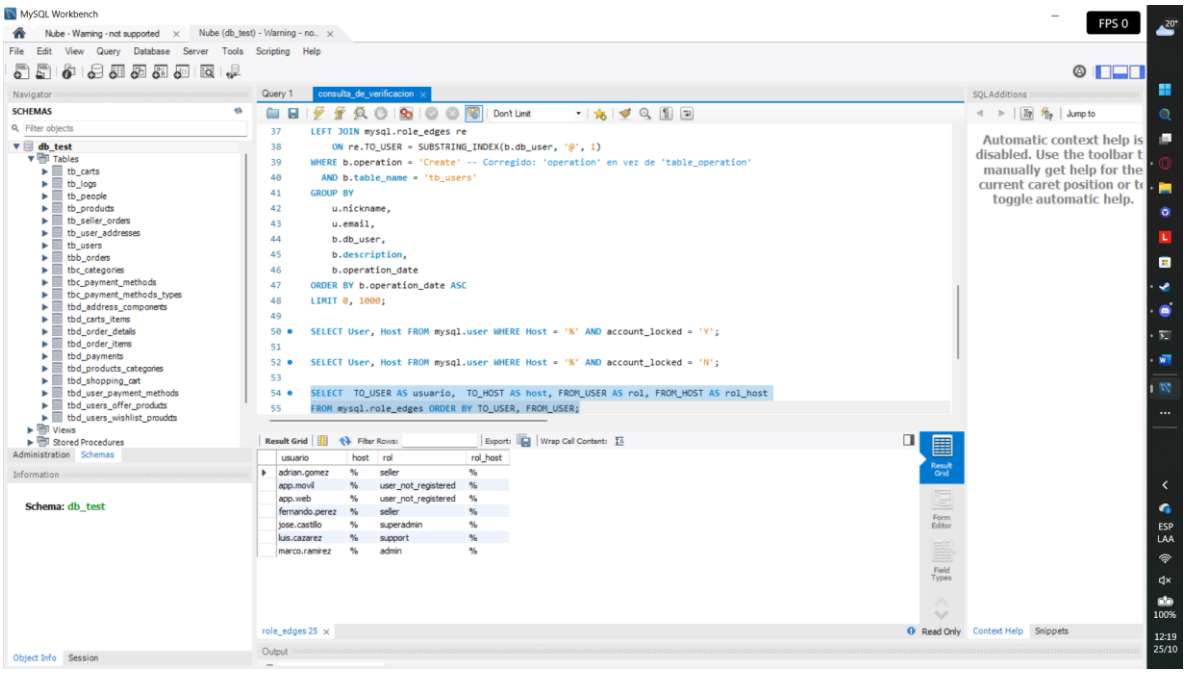
SELECT User, Host FROM mysql.user WHERE Host = '%' AND account_locked = 'Y';



SELECT User, Host FROM mysql.user WHERE Host = '%' AND account_locked = 'N';



SELECT TO_USER AS usuario, TO_HOST AS host, FROM_USER AS rol, FROM_HOST AS rol_host FROM mysql.role_edges ORDER BY TO_USER, FROM_USER;



Conclusiones

Competencias desarrolladas. Aprendí a desplegar un entorno completo de bases de datos con Docker Compose, a configurar volúmenes persistentes, puertos y comprobaciones de salud, a administrar contenedores de un VPS compartido sin afectar a otros usuarios, y a migrar una base de datos de MySQL a PostgreSQL, incluyendo tablas, triggers, vistas y procedimientos.

Dificultades técnicas y soluciones.

- *Puertos:* al principio confundí el puerto con mi usuario y elegí números fuera del rango válido. Entendí que dos servicios no pueden escuchar en el mismo puerto y que pgAdmin se conecta a PostgreSQL por la red interna de Docker (host postgres, puerto 5432). La lista de verificación fijó PostgreSQL en 40628 y MySQL en 40629.
- *Permisos del sistema:* sudo rechazó mi cuenta porque no pertenece al grupo sudo. Pude seguir con Docker, que sí tengo habilitado, y solicité el permiso al administrador del VPS.
- *Respaldo de MySQL:* el primer intento falló con el error 1045 porque docker exec necesita la opción -i para recibir datos y la clave no llegó. Después el error

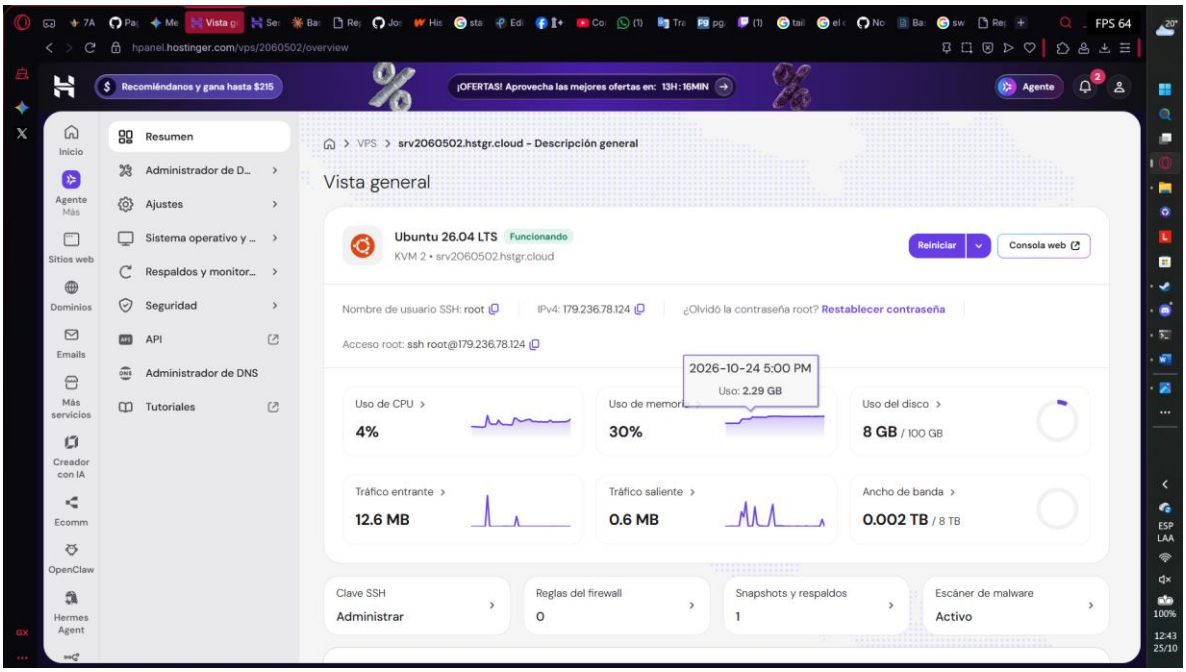
6125 por diferencias entre MySQL 8.0 y 8.4, que resolví desactivando la restricción de llaves foráneas solo durante la restauración.

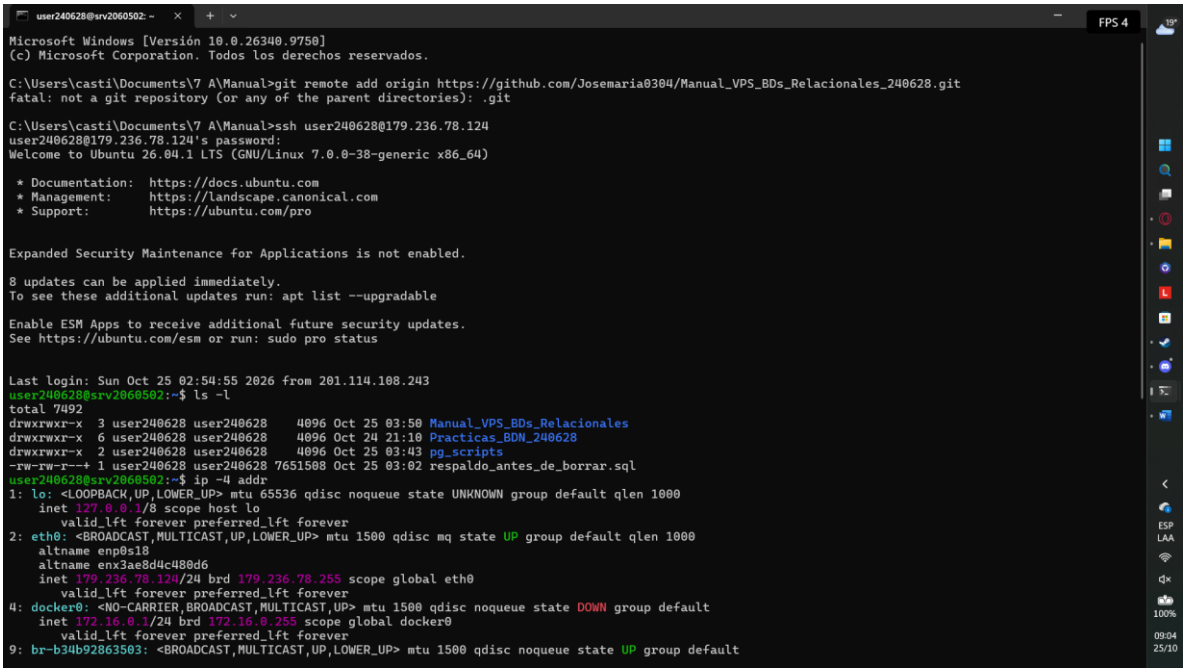
- *Migración a PostgreSQL:* hubo que reescribir tipos, triggers y procedimientos, y cambiar las llaves foráneas que apuntaban a columnas no únicas. Cargar los scripts en el orden correcto evitó duplicar la bitácora.

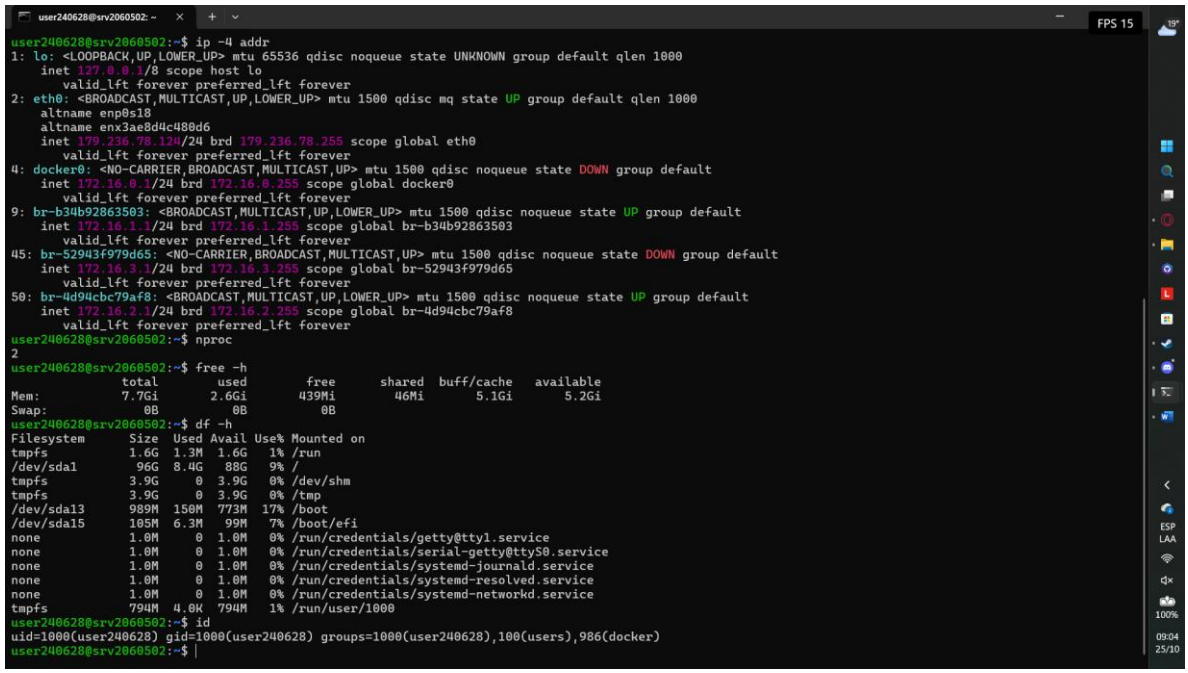
Trabajar con bases de datos en la nube exige cuidar mucho la seguridad (credenciales fuera del repositorio, puertos y firewall), la reproducibilidad (todo definido en un docker-compose.yml) y los respaldos antes de borrar o recrear servicios ya puestos en el VPS. Contar con scripts y evidencias ordenadas facilita repetir el despliegue y comprobaciones del trabajo.

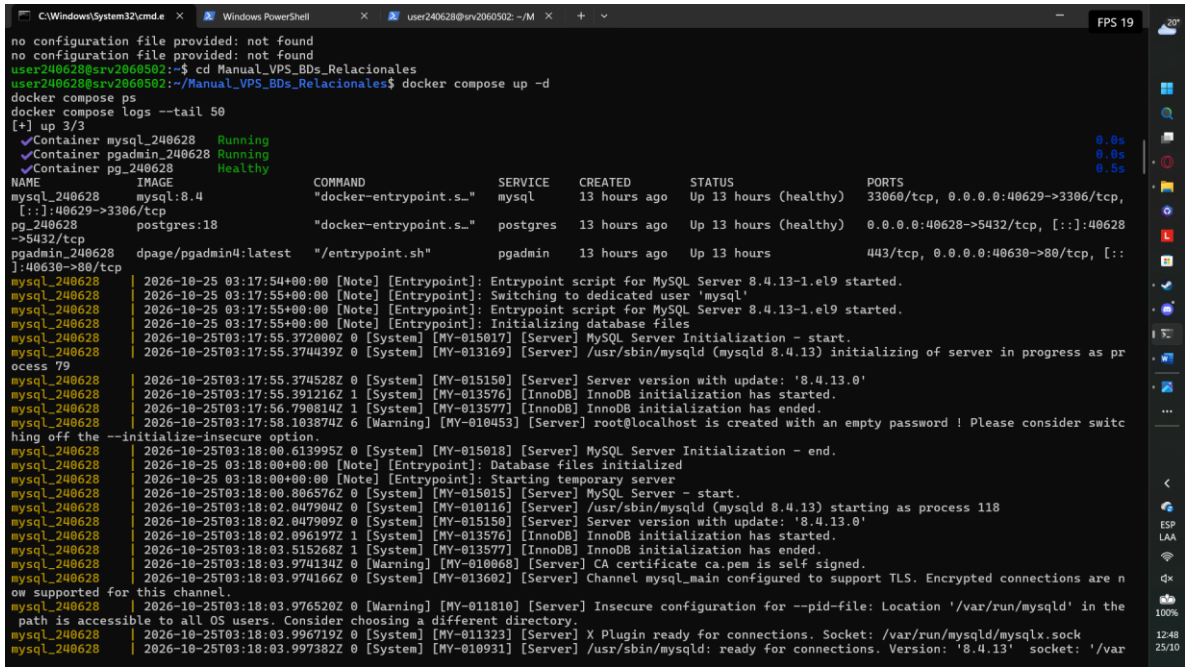
Link PgAdmin: http://srv2060502.hstgr.cloud:40630/login?next=/browser/

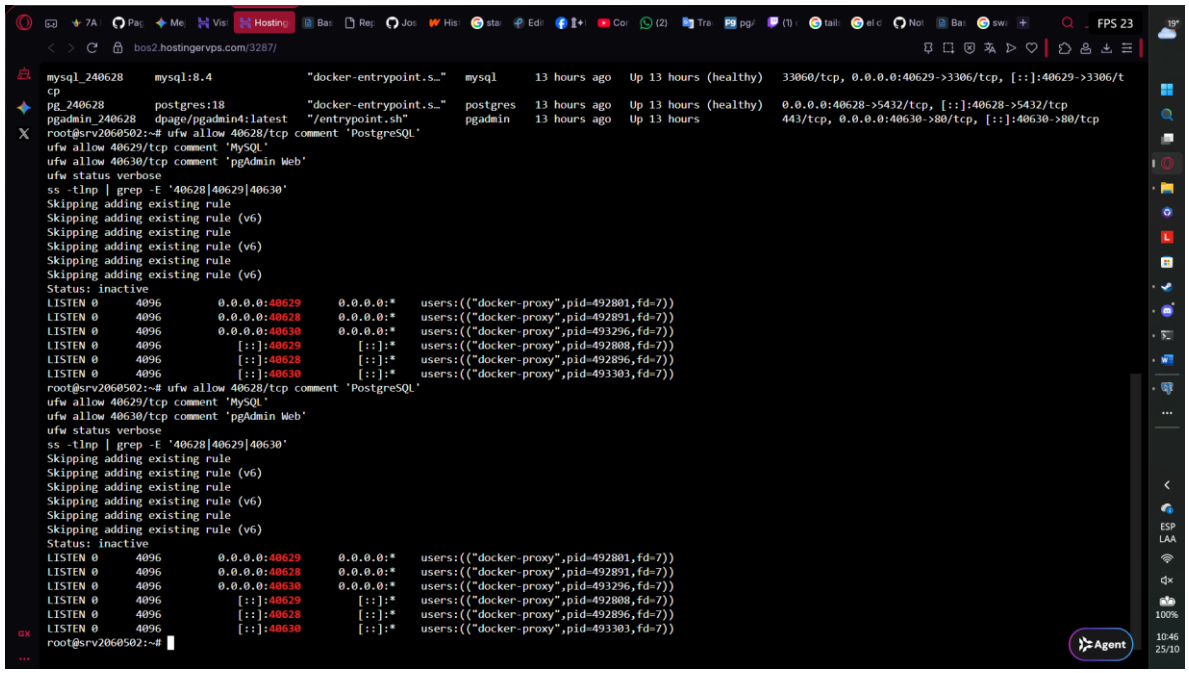
EVIDENCIAS GENERALES

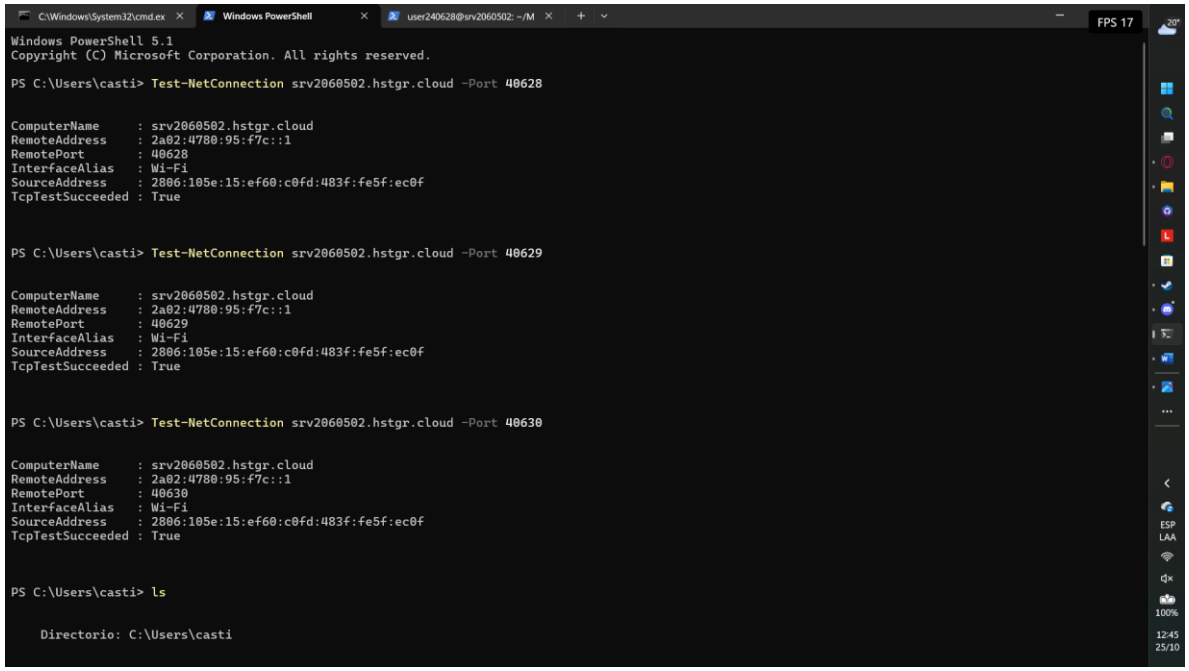


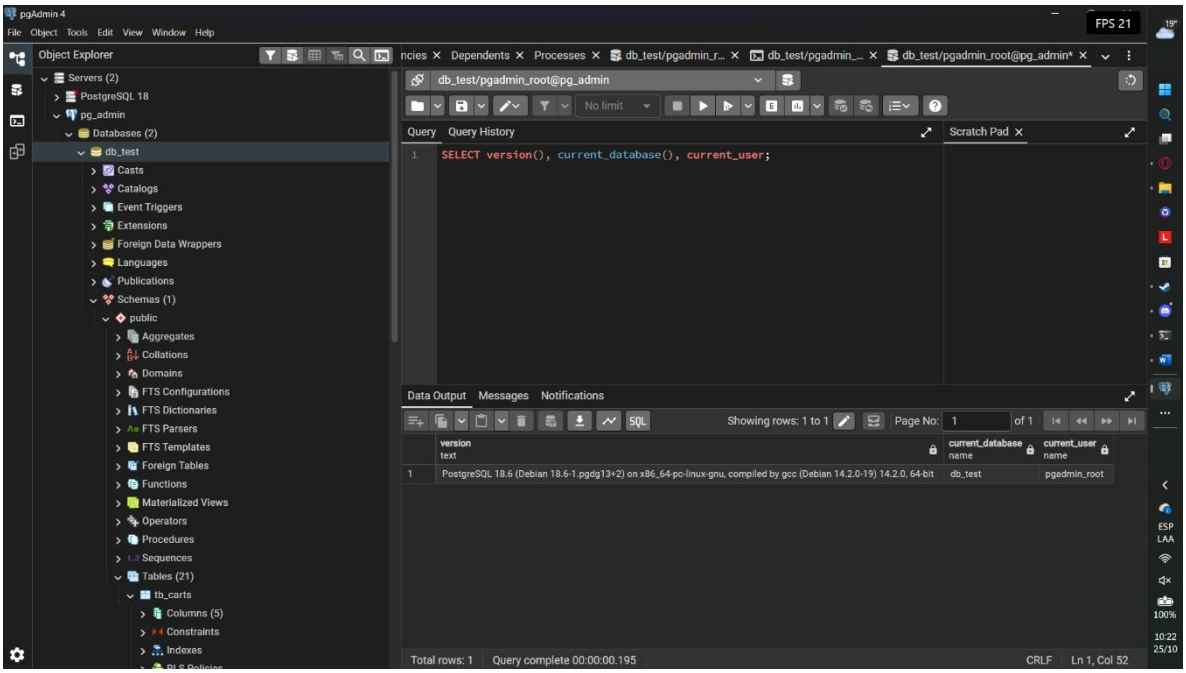


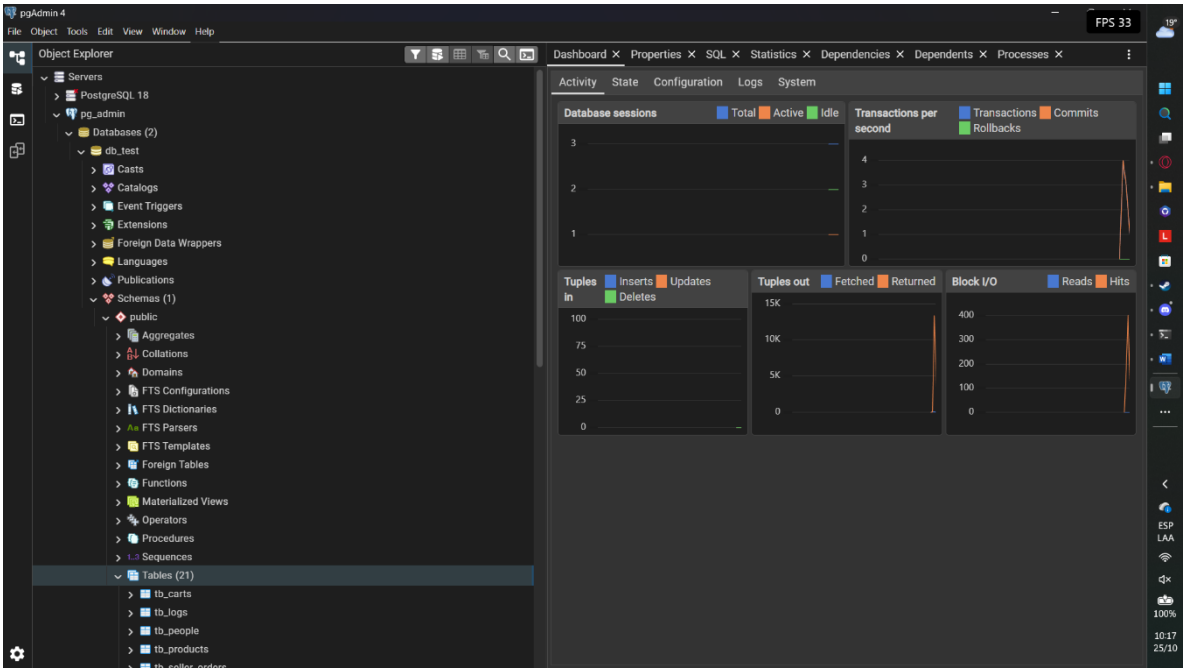

user240628@srv2060502: ~
FPS 15
user240628@srv2060502:~$ ip -4 addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
inet 127.0.0.1/8 scope host lo
valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
altname enp0s18
altname enx3ae8d4c480d6
inet 179.236.78.124/24 brd 179.236.78.255 scope global eth0
valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
inet 172.16.0.1/24 brd 172.16.0.255 scope global docker0
valid_lft forever preferred_lft forever
9: br-b34b92863503: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
inet 172.16.1.1/24 brd 172.16.1.255 scope global br-b34b92863503
valid_lft forever preferred_lft forever
45: br-52943f979d65: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
inet 172.16.3.1/24 brd 172.16.3.255 scope global br-52943f979d65
valid_lft forever preferred_lft forever
50: br-4d94cbc79af8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
inet 172.16.2.1/24 brd 172.16.2.255 scope global br-4d94cbc79af8
valid_lft forever preferred_lft forever
user240628@srv2060502:~$ nproc
2
user240628@srv2060502:~$ free -h
total used free shared buff/cache available
Mem: 7.7Gi 2.6Gi 439Mi 46Mi 5.1Gi 5.2Gi
Swap: 0B 0B 0B
user240628@srv2060502:~$ df -h
Filesystem Size Used Avail Use% Mounted on
tmpfs 1.6G 1.3M 1.6G 1% /run
/dev/sda1 96G 8.4G 88G 9% /
tmpfs 3.9G 0 3.9G 0% /dev/shm
tmpfs 3.9G 0 3.9G 0% /tmp
/dev/sda13 989M 150M 773M 17% /boot
/dev/sda15 105M 6.3M 99M 7% /boot/efi
none 1.0M 0 1.0M 0% /run/credentials/getty@tty1.service
none 1.0M 0 1.0M 0% /run/credentials/serial-getty@ttyS0.service
none 1.0M 0 1.0M 0% /run/credentials/systemd-journald.service
none 1.0M 0 1.0M 0% /run/credentials/systemd-resolved.service
none 1.0M 0 1.0M 0% /run/credentials/systemd-networkd.service
tmpfs 794M 4.0K 794M 1% /run/user/1000
user240628@srv2060502:~$ id
uid=1000(user240628) gid=1000(user240628) groups=1000(user240628),100(users),986(docker)
user240628@srv2060502:~$

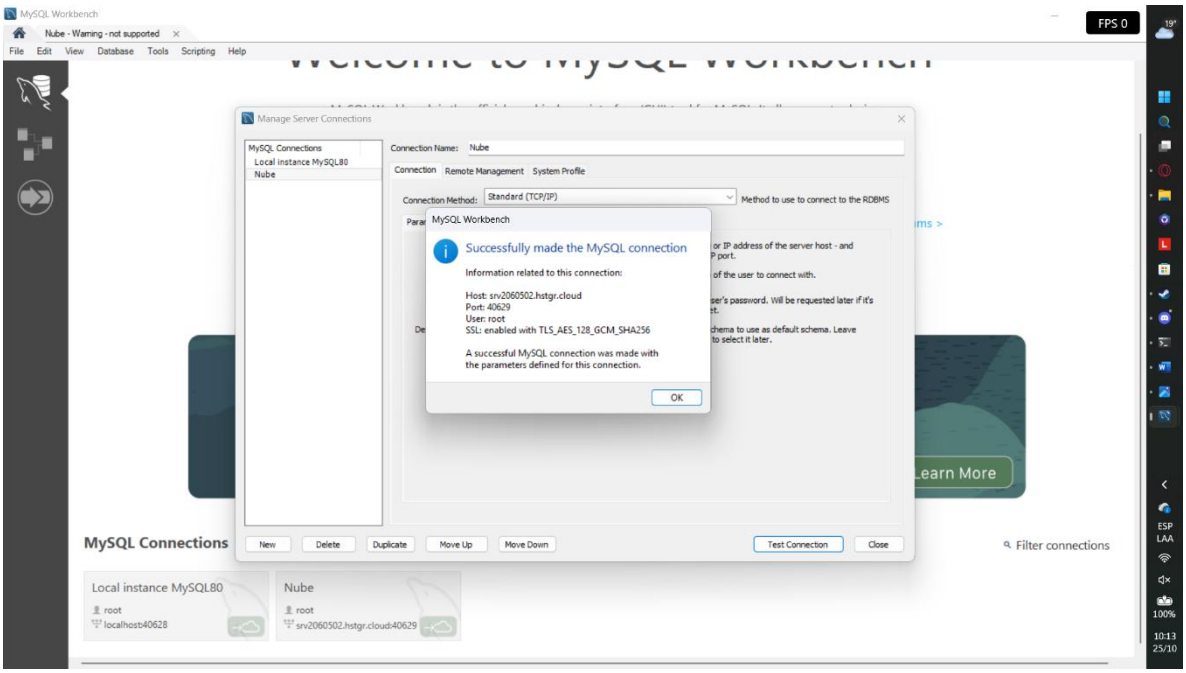

C:\Windows\System32\cmd.e
Windows PowerShell
user240628@srv2060502: ~/M
FPS 19
no configuration file provided: not found
no configuration file provided: not found
user240628@srv2060502:~$ cd Manual_VPS_BDs_Relacionales
user240628@srv2060502:~/Manual_VPS_BDs_Relacionales$ docker compose up -d
docker compose ps
docker compose logs --tail 50
[+] up 3/3
Container mysql_240628 Running 0.0s
Container pgadmin_240628 Running 0.0s
Container pg_240628 Healthy 0.5s
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
mysql_240628 mysql:8.4 "docker-entrypoint.s…" mysql 13 hours ago Up 13 hours (healthy) 33060/tcp, 0.0.0.0:40629->3306/tcp, [::]:40629->3306/tcp
pg_240628 postgres:18 "docker-entrypoint.s…" postgres 13 hours ago Up 13 hours (healthy) 0.0.0.0:40628->5432/tcp, [::]:40628->5432/tcp
pgadmin_240628 dpage/pgadmin4:latest "/entrypoint.sh" pgadmin 13 hours ago Up 13 hours 443/tcp, 0.0.0.0:40630->80/tcp, [::]:40630->80/tcp
mysql_240628 | 2026-10-25 03:17:54+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.13-1.el9 started.
mysql_240628 | 2026-10-25 03:17:55+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
mysql_240628 | 2026-10-25 03:17:55+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.13-1.el9 started.
mysql_240628 | 2026-10-25 03:17:55+00:00 [Note] [Entrypoint]: Initializing database files
mysql_240628 | 2026-10-25T03:17:55.372000Z 0 [System] [MY-015017] [Server] MySQL Server Initialization - start.
mysql_240628 | 2026-10-25T03:17:55.374439Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.4.13) initializing of server in progress as process 79
mysql_240628 | 2026-10-25T03:17:55.374528Z 0 [System] [MY-015150] [Server] Server version with update: '8.4.13.0'
mysql_240628 | 2026-10-25T03:17:55.391216Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
mysql_240628 | 2026-10-25T03:17:56.790814Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
mysql_240628 | 2026-10-25T03:17:58.103874Z 6 [Warning] [MY-010453] [Server] root@localhost is created with an empty password ! Please consider switching off the --initialize-insecure option.
mysql_240628 | 2026-10-25T03:18:00.613995Z 0 [System] [MY-015018] [Server] MySQL Server Initialization - end.
mysql_240628 | 2026-10-25 03:18:00+00:00 [Note] [Entrypoint]: Database files initialized
mysql_240628 | 2026-10-25 03:18:00+00:00 [Note] [Entrypoint]: Starting temporary server
mysql_240628 | 2026-10-25T03:18:00.806576Z 0 [System] [MY-015015] [Server] MySQL Server - start.
mysql_240628 | 2026-10-25T03:18:02.047904Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.4.13) starting as process 118
mysql_240628 | 2026-10-25T03:18:02.047909Z 0 [System] [MY-015150] [Server] Server version with update: '8.4.13.0'
mysql_240628 | 2026-10-25T03:18:02.096197Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
mysql_240628 | 2026-10-25T03:18:03.515268Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
mysql_240628 | 2026-10-25T03:18:03.974134Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
mysql_240628 | 2026-10-25T03:18:03.974166Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported for this channel.
mysql_240628 | 2026-10-25T03:18:03.976520Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a different directory.
mysql_240628 | 2026-10-25T03:18:03.996719Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Socket: /var/run/mysqld/mysqlx.sock
mysql_240628 | 2026-10-25T03:18:03.997382Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.4.13' socket: '/var

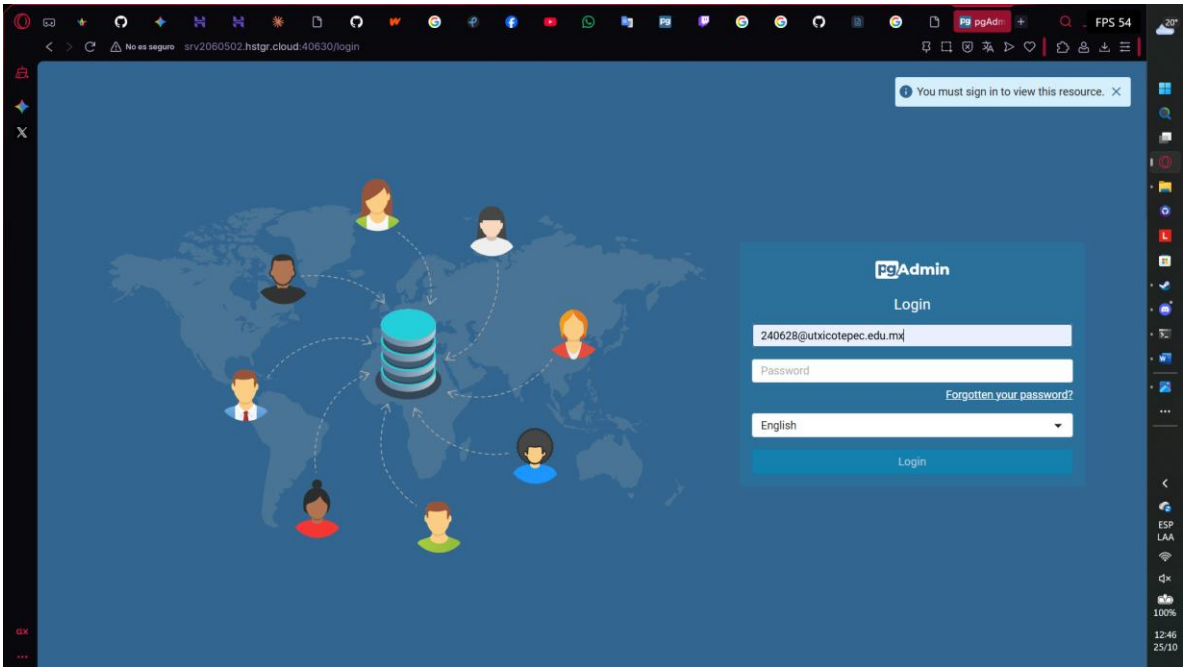

bos2.hostingervps.com/3287/
mysql_240628 mysql:8.4 "docker-entrypoint.s…" mysql 13 hours ago Up 13 hours (healthy) 33060/tcp, 0.0.0.0:40629->3306/tcp, [::]:40629->3306/tcp
pg_240628 postgres:18 "docker-entrypoint.s…" postgres 13 hours ago Up 13 hours (healthy) 0.0.0.0:40628->5432/tcp, [::]:40628->5432/tcp
pgadmin_240628 dpage/pgadmin4:latest "/entrypoint.sh" pgadmin 13 hours ago Up 13 hours 443/tcp, 0.0.0.0:40630->80/tcp, [::]:40630->80/tcp
root@srv2060502:~# ufw allow 40628/tcp comment 'PostgreSQL'
ufw allow 40629/tcp comment 'MySQL'
ufw allow 40630/tcp comment 'pgAdmin Web'
ufw status verbose
ss -tlnp | grep -E '40628|40629|40630'
Skipping adding existing rule
Skipping adding existing rule (v6)
Skipping adding existing rule
Skipping adding existing rule (v6)
Skipping adding existing rule
Skipping adding existing rule (v6)
Status: inactive
LISTEN 0 4096 0.0.0.0:40629 0.0.0.0:* users:(("docker-proxy",pid=492801,fd=7))
LISTEN 0 4096 0.0.0.0:40628 0.0.0.0:* users:(("docker-proxy",pid=492891,fd=7))
LISTEN 0 4096 0.0.0.0:40630 0.0.0.0:* users:(("docker-proxy",pid=493296,fd=7))
LISTEN 0 4096 [::]:40629 [::]:* users:(("docker-proxy",pid=492808,fd=7))
LISTEN 0 4096 [::]:40628 [::]:* users:(("docker-proxy",pid=492896,fd=7))
LISTEN 0 4096 [::]:40630 [::]:* users:(("docker-proxy",pid=493303,fd=7))
root@srv2060502:~# ufw allow 40628/tcp comment 'PostgreSQL'
ufw allow 40629/tcp comment 'MySQL'
ufw allow 40630/tcp comment 'pgAdmin Web'
ufw status verbose
ss -tlnp | grep -E '40628|40629|40630'
Skipping adding existing rule
Skipping adding existing rule (v6)
Skipping adding existing rule
Skipping adding existing rule (v6)
Skipping adding existing rule
Skipping adding existing rule (v6)
Status: inactive
LISTEN 0 4096 0.0.0.0:40629 0.0.0.0:* users:(("docker-proxy",pid=492801,fd=7))
LISTEN 0 4096 0.0.0.0:40628 0.0.0.0:* users:(("docker-proxy",pid=492891,fd=7))
LISTEN 0 4096 0.0.0.0:40630 0.0.0.0:* users:(("docker-proxy",pid=493296,fd=7))
LISTEN 0 4096 [::]:40629 [::]:* users:(("docker-proxy",pid=492808,fd=7))
LISTEN 0 4096 [::]:40628 [::]:* users:(("docker-proxy",pid=492896,fd=7))
LISTEN 0 4096 [::]:40630 [::]:* users:(("docker-proxy",pid=493303,fd=7))
root@srv2060502:~#
Agent


C:\Windows\System32\cmd.ex
Windows PowerShell
user240628@srv2060502: ~/M
Windows PowerShell 5.1
Copyright (C) Microsoft Corporation. All rights reserved.
PS C:\Users\casti> Test-NetConnection srv2060502.hstgr.cloud -Port 40628
ComputerName : srv2060502.hstgr.cloud
RemoteAddress : 2a02:4780:95:f7c::1
RemotePort : 40628
InterfaceAlias : Wi-Fi
SourceAddress : 2806:105e:15:ef60:c0fd:483f:fe5f:ec0f
TcpTestSucceeded : True
PS C:\Users\casti> Test-NetConnection srv2060502.hstgr.cloud -Port 40629
ComputerName : srv2060502.hstgr.cloud
RemoteAddress : 2a02:4780:95:f7c::1
RemotePort : 40629
InterfaceAlias : Wi-Fi
SourceAddress : 2806:105e:15:ef60:c0fd:483f:fe5f:ec0f
TcpTestSucceeded : True
PS C:\Users\casti> Test-NetConnection srv2060502.hstgr.cloud -Port 40630
ComputerName : srv2060502.hstgr.cloud
RemoteAddress : 2a02:4780:95:f7c::1
RemotePort : 40630
InterfaceAlias : Wi-Fi
SourceAddress : 2806:105e:15:ef60:c0fd:483f:fe5f:ec0f
TcpTestSucceeded : True
PS C:\Users\casti> ls
Directorio: C:\Users\casti


pgAdmin 4
File Object Tools Edit View Window Help
FPS 21
Object Explorer
Servers (2)
PostgreSQL 18
pg_admin
Databases (2)
db_test
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (21)
tb_carts
Columns (5)
Constraints
Indexes
Dependents
Processes
db_test/pgadmin_root@pg_admin*
db_test/pgadmin_root@pg_admin
No limit
Query
Query History
Scratch Pad
SELECT version(), current_database(), current_user;
Data Output
Messages
Notifications
Showing rows: 1 to 1
Page No: 1 of 1
version
text
current_database
name
current_user
name
PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
db_test
pgadmin_root
Total rows: 1
Query complete 00:00:00.195
CRLF
Ln 1, Col 52


pgAdmin 4
File Object Tools Edit View Window Help
FPS 33
Object Explorer
Servers
PostgreSQL 18
pg_admin
Databases (2)
db_test
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (21)
tb_carts
tb_logs
tb_people
tb_products
Dashboard
Properties
SQL
Statistics
Dependencies
Dependents
Processes
Activity
State
Configuration
Logs
System
Database sessions
Total
Active
Idle
Transactions per second
Transactions
Commits
Rollbacks
Tuples in
Inserts
Updates
Deletes
Tuples out
Fetched
Returned
Block I/O
Reads
Hits


MySQL Workbench
Nube - Warning - not supported
File Edit View Database Tools Scripting Help
FPS 0
Manage Server Connections
MySQL Connections
Local instance MySQL80
Nube
Connection Name: Nube
Connection Remote Management System Profile
Connection Method: Standard (TCP/IP)
Method to use to connect to the RDBMS
MySQL Workbench
Successfully made the MySQL connection
Information related to this connection:
Host: srv2060502.hstgr.cloud
Port: 40629
User: root
SSL: enabled with TLS_AES_128_GCM_SHA256
A successful MySQL connection was made with the parameters defined for this connection.
OK
Learn More
MySQL Connections
New Delete Duplicate Move Up Move Down Test Connection Close
Filter connections
Local instance MySQL80
root
localhost:40628
Nube
root
srv2060502.hstgr.cloud:40629


No es seguro
srv2060502.hstgr.cloud:40630/login
FPS 54
You must sign in to view this resource.
pgAdmin
Login
240628@utxicotepec.edu.mx
Password
Forgotten your password?
English
Login

Manual

PostgreSQL es un sistema de gestión de bases de datos relacionales y de código abierto. Se destaca por su robustez, rendimiento y soporte avanzado para consultas SQL completas y herramientas de gestión gráfica como pgAdmin 4.

Este manual/guía se enfoca en el proceso paso a paso para la descarga, selección de componentes, asignación de parámetros de red/seguridad y verificación de PostgreSQL en un sistema operativo Windows (específicamente optimizado para Windows 11 de 64 bits).

Requerimientos para instalarlo

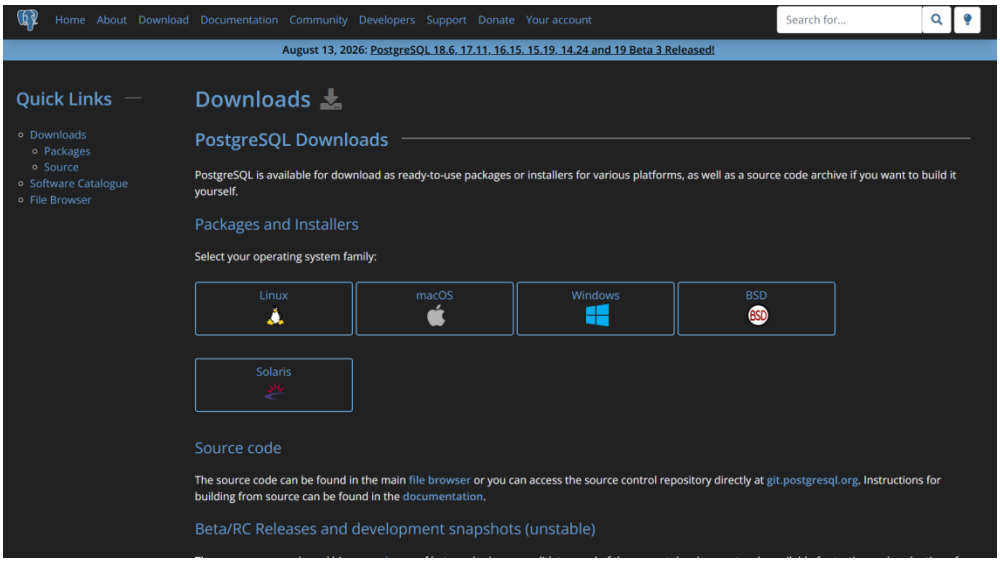
Para llevar a cabo la instalación descrita en el documento, se requieren los siguientes elementos mínimos del sistema y de software:

- **Sistema Operativo:** Windows 10/11 de 64 bits (x86-64).
- **Archivo de Instalación:** Instalador ejecutable descargado desde el sitio web oficial de PostgreSQL / EDB (por ejemplo, postgresql-16.1-1-windows-x64.exe o equivalente de la versión seleccionada).
- **Conexión a Internet:** Requerida durante la instalación si se desea utilizar la herramienta **Stack Builder** para descargar complementos o *drivers* adicionales (como psqlODBC).
- **Privilegios de Administrador:** Permisos del sistema operativo para instalar servicios e interactuar con el directorio C:\Program Files.
- **Permisos de Red:** Un puerto TCP disponible para la escucha del servicio de base de datos (por defecto 5432 o personalizado).

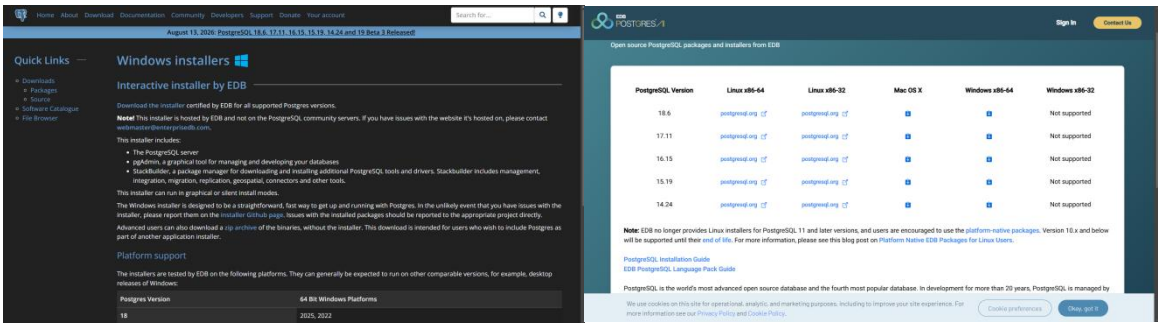
Pasos de Instalacion

Lo primero que debemos de realizar para la instalación de PostgreSQL es descargar el archivo necesario para llevar a cabo la instalación, éste deberá ser compatible con el sistema operativo de nuestro equipo de cómputo. Este manual será enfocado a la última versión de Windows 11.

1. Accedemos a la página Oficial de ProgreSql. Y seleccionamos el Sistema operativo.



2. Una vez seleccionado el Sistema presionamos en la opción marcada de “Download Installer” nos llevara a este apartado. Seleccionen la versión de Windows x86-64 o x86-32 dependiendo del dispositivo.



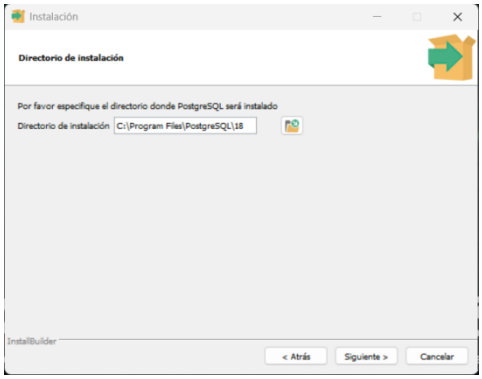
3. El Instalador debe mostrarse de la siguiente manera



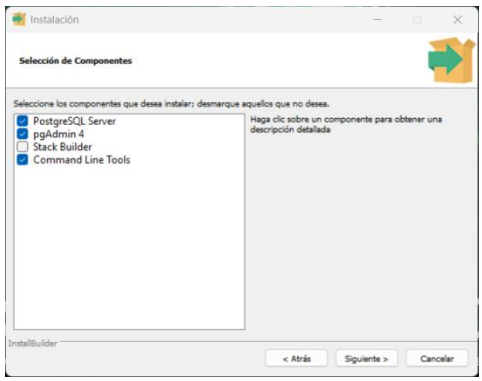
4. Deberemos ejecutar el archivo, y se mostrará la siguiente ventana: Donde tendremos que hacer clic en "Siguiente".



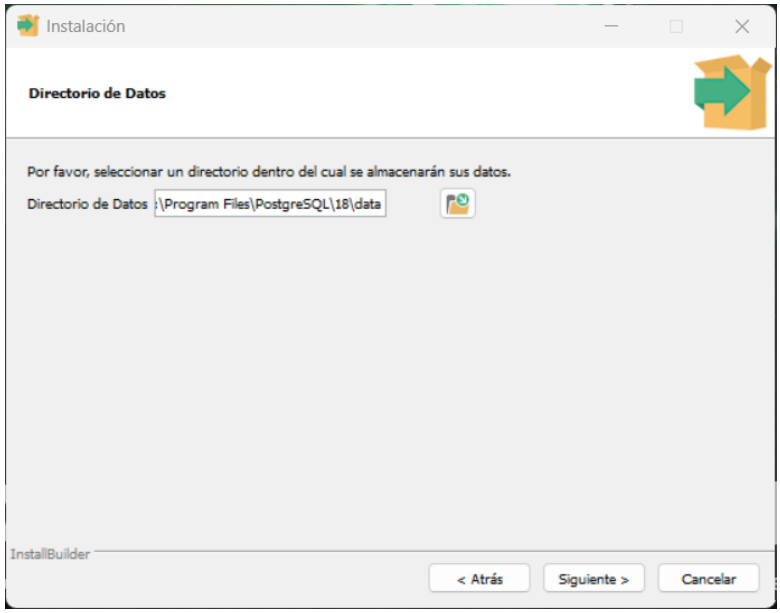
5. En esta parte se mostrará la ruta de instalación. Puedes cambiar la ruta a donde quieras instalarlo. Pero lo mas recomendable es como ya viene establecido. De hay le dan en “Siguiente”.



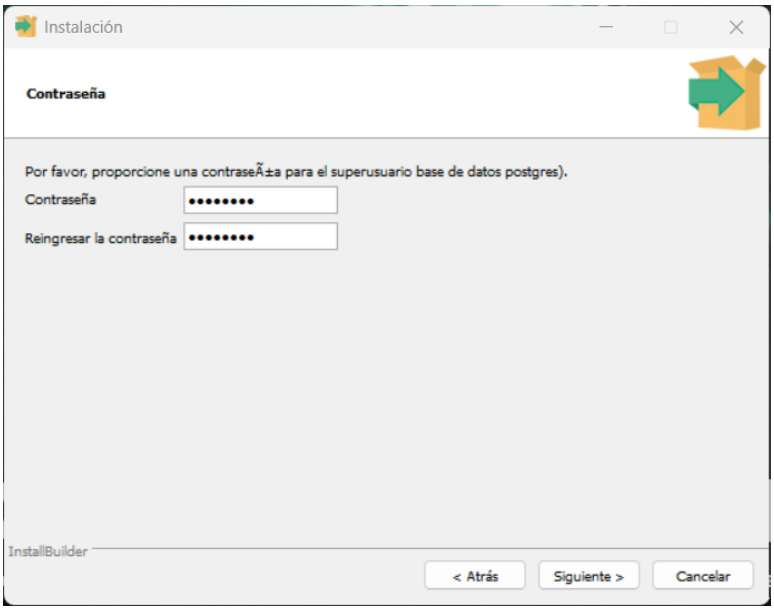
6. En esta parte solo vamos a tener tres casillas seleccionada “ProgreSQL Server”, “pgAdmin 4”, “Command Line Tools”. Una vez seleccionadas, le damos en siguiente.



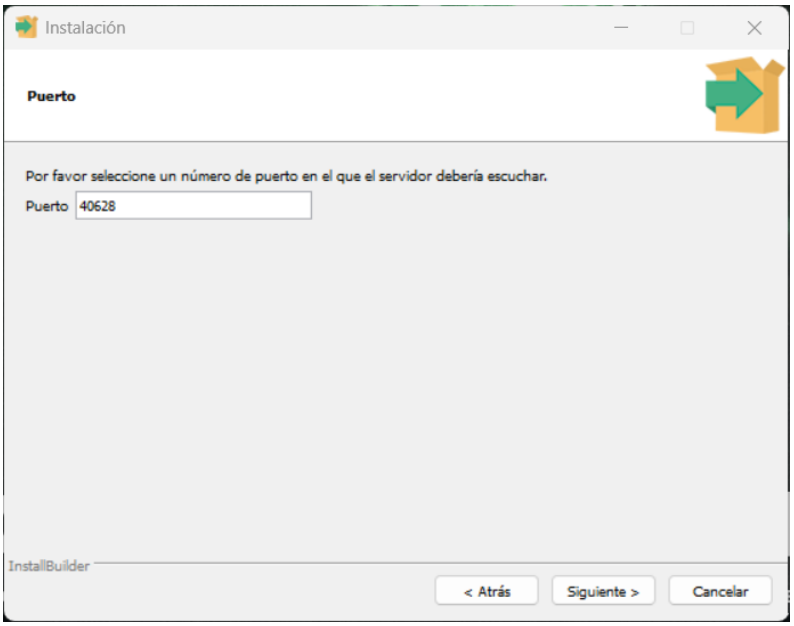
7. En esta parte se desplegará la siguiente ventana, en la cual deberemos seleccionar donde se almacenarán los datos, siendo recomendable utilizar la ruta por defecto. Después, procederemos a hacer clic en "Instalar". Puede elegir otra ubicación sin embargo es recomendable recordar para futuros trabajos de sus tablas, por ejemplo para realizar algun respaldo o levantar una base de datos externa... deberá recordar la ubicación.



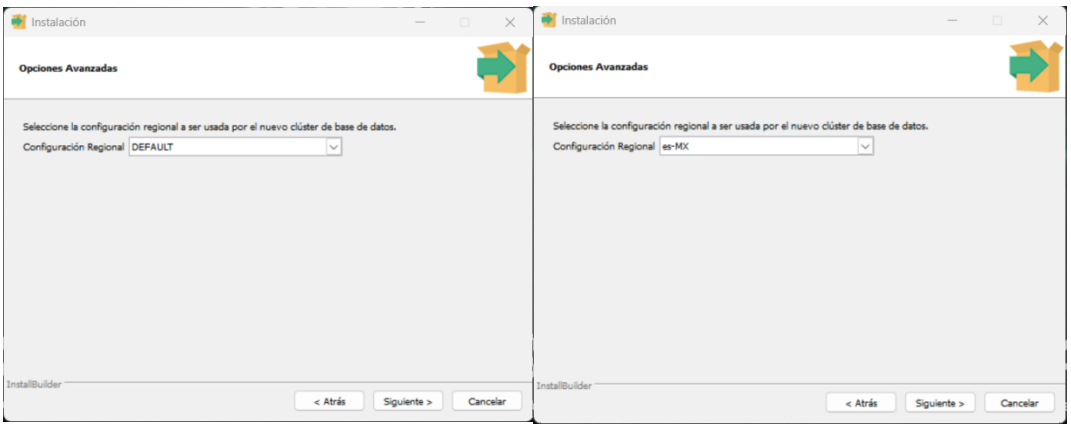
8. Continuando con la configuración, hay que asignar una contraseña. Una vez pensada la contraseña le dan en siguiente.



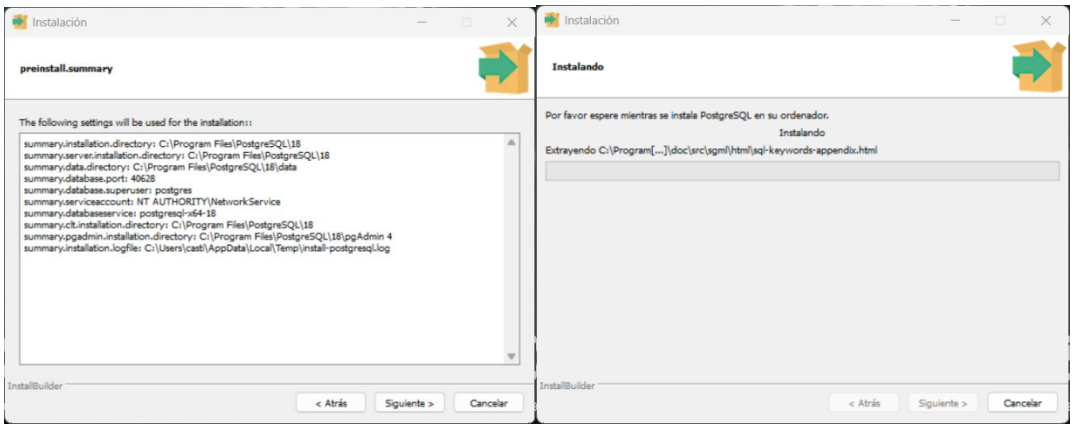
9. En esta pantalla de configuración, es necesario mantener los valores por defecto para garantizar el correcto funcionamiento de nuestro servidor. Que son el 5432. (40628 es mi Matricula que usare como puerto).



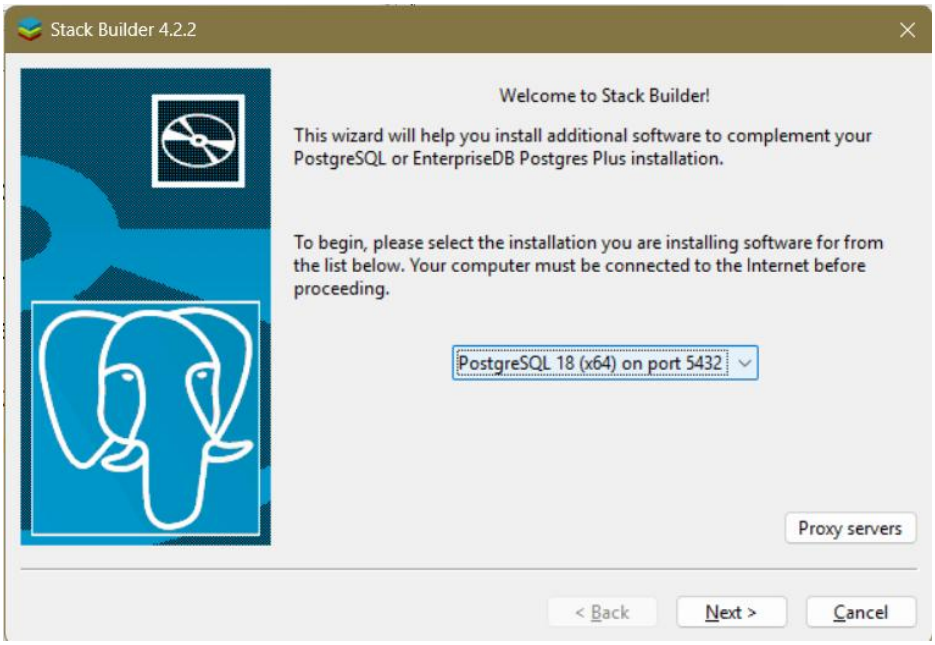
10. En la parte de opciones avanzadas lo mas recomendable es dejarlo como “Default”. Si vas a manejar datos como por ejemplo en español puedes colocarlo en la es-MX o dependiendo de tu región, pero esto cambiara la forma en que introduzcas los datos (DD/MM/AAAA vs MM/DD/AAAA), ordenamiento y la compatibilidad.



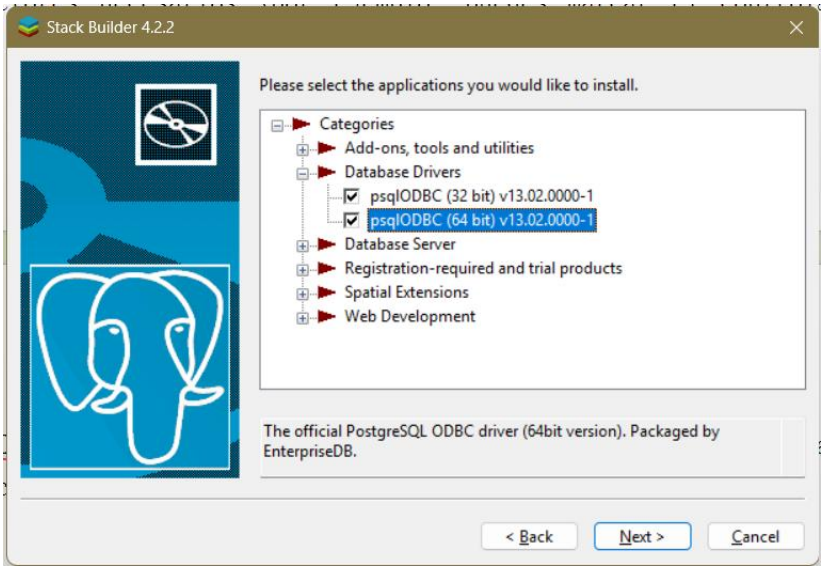
11. Se desplegará la siguiente ventana donde solamente tendremos que dar clic en "Siguiente". He iniciar la instalación dándole al Boton “Siguiente”



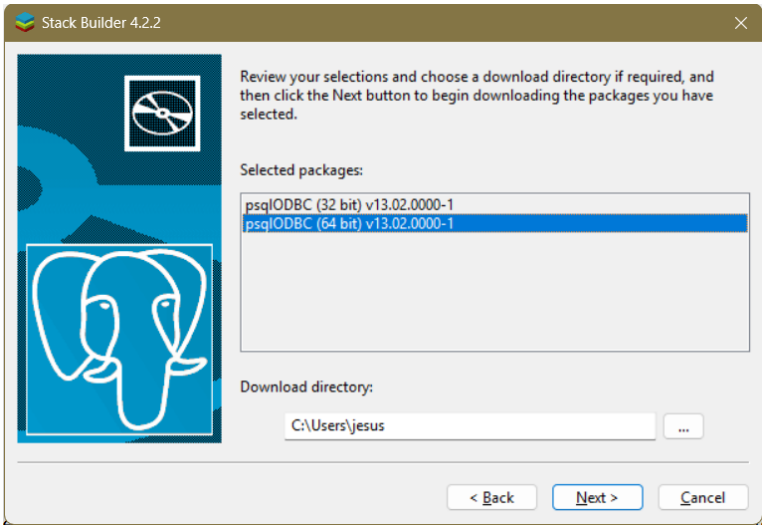
12. Al aparecer la ventana de **Stack Builder**, despliega el menú desplegable para seleccionar tu instalación local de PostgreSQL (en este caso, **PostgreSQL 18 (x64) on port 5432** o el puerto que hayas configurado). Una vez seleccionada la opción correcta, haz clic en **"Next >"** para continuar con el asistente.



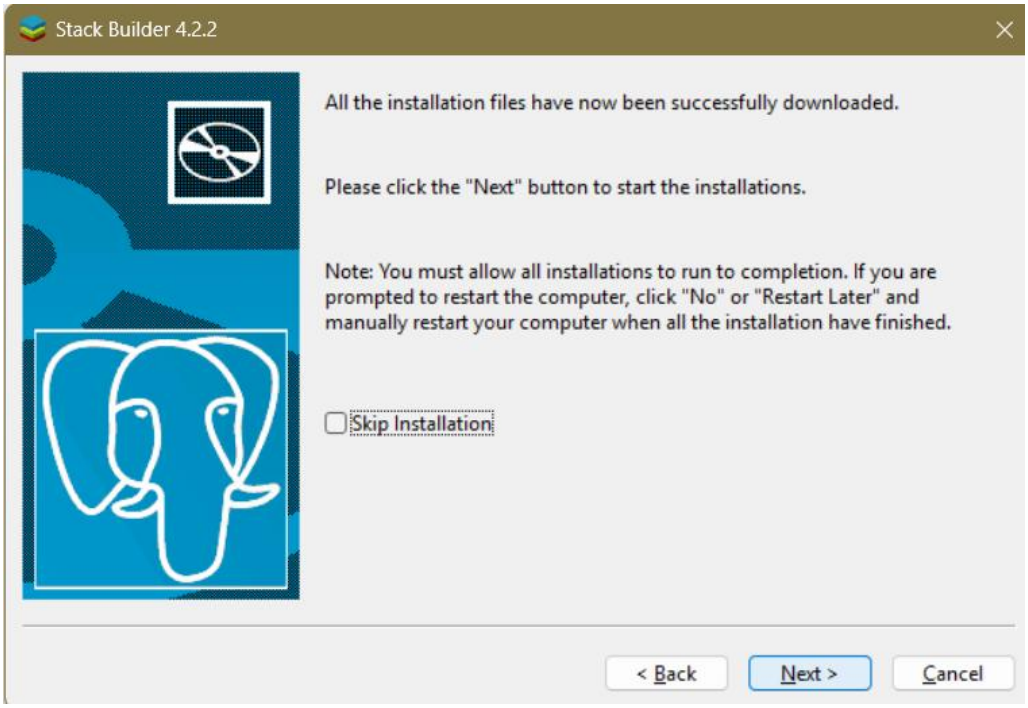
13. En esta sección de **Stack Builder**, el asistente muestra las categorías de herramientas y complementos adicionales que puedes instalar. Expande la categoría **"Database Drivers"** y selecciona los conectores necesarios (por ejemplo, puedes marcar el controlador **psqlODBC** de 64 bits para futuras conexiones). Una vez seleccionados los componentes deseados, haz clic en **"Next >"** para continuar.



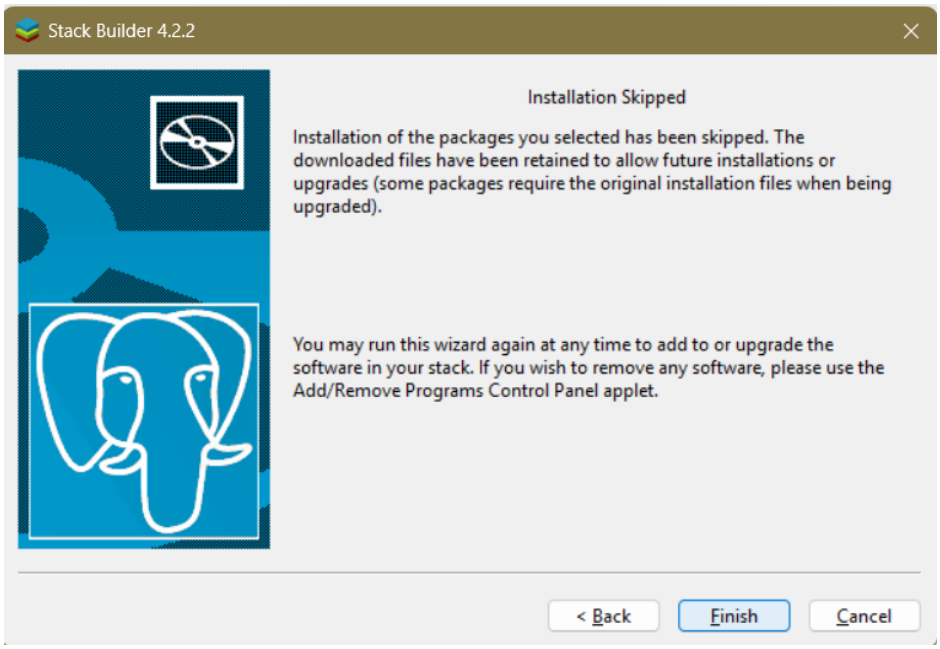
14.En esta ventana se muestra un resumen de los paquetes seleccionados para descargar, junto con la ruta temporal donde se guardarán (*Download directory*). Revisa que todo esté correcto y haz clic en **"Next >"** para iniciar la descarga de los archivos seleccionados.



15.Una vez completada la descarga de los archivos seleccionados, el asistente indicará que todo se ha descargado exitosamente. Haz clic en **"Next >"** para comenzar con la instalación de los componentes adicionales.



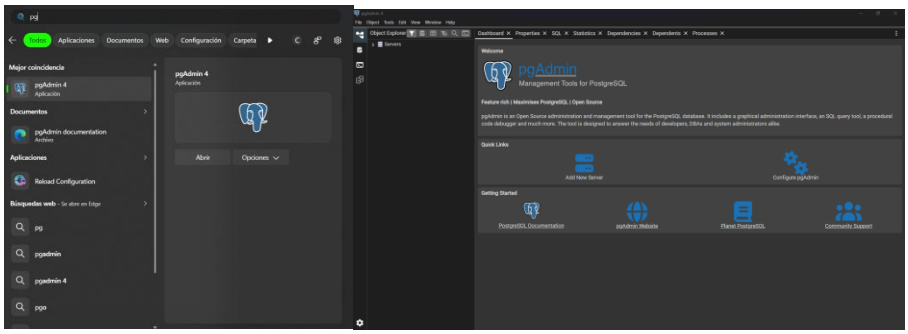
16.Esta ventana indica que la instalación de los paquetes adicionales ha finalizado o se ha omitido según las opciones elegidas. Por último, haz clic en el botón **"Finish"** para cerrar por completo el asistente de Stack Builder y concluir todo el proceso de instalación de PostgreSQL.



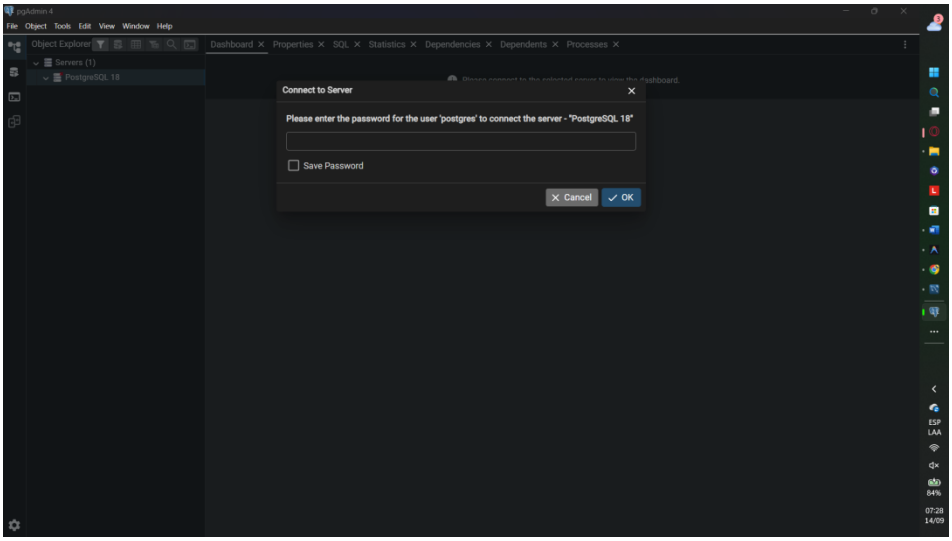
17.Una vez finalizado el proceso, se mostrará la siguiente ventana donde tendremos que dar clic en 'Terminar'. ¡Y listo, ya tendrás el PostgreSQL, listo para ejecutar!



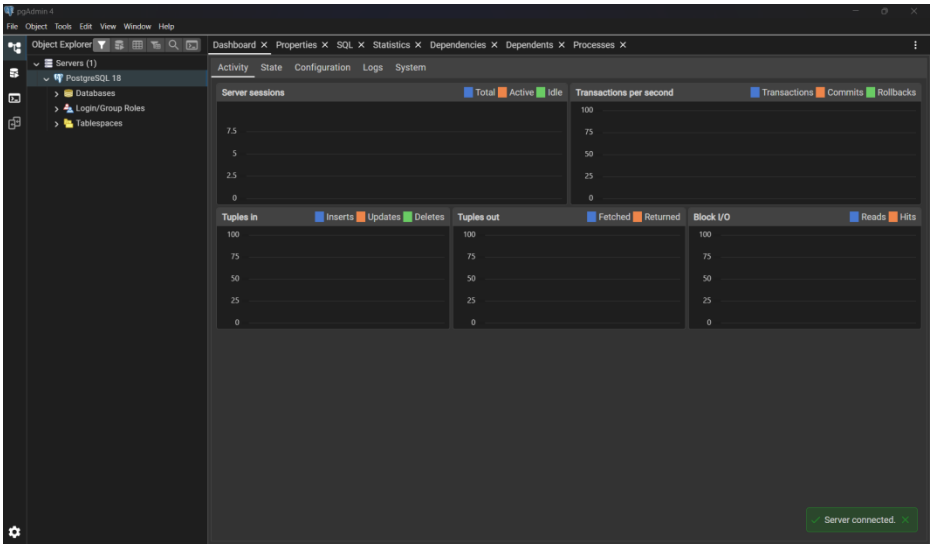
18.Verificamos que este instalado, lo abrimos.



19.Introducimos la contraseña que recientemente agregamos.



20. Una vez completado el proceso de inicio de sesión, se desplegará la siguiente pantalla, confirmando el acceso exitoso apostgresql.



Razones por las que se configura así:

Durante el proceso del instalador, se solicitan distintos parámetros con justificaciones técnicas específicas:

1. **Ruta de Instalación predeterminada (C:\Program Files\PostgreSQL\...):**

Es recomendable mantener la ubicación por defecto para evitar problemas de permisos de usuario o incompatibilidades en las rutas ejecutables de los servicios de Windows.

2. **Selección de Componentes (Servidor, pgAdmin 4 y Command Line Tools):**

Se eligen **PostgreSQL Server** (el motor principal de la base de datos), **pgAdmin 4** (la interfaz gráfica indispensable para administrar el motor sin comandos complejos) y **Command Line Tools** (herramientas de consola indispensables para scripts o respaldos). Se desmarca **Stack Builder** si no se requieren controladores inmediatos para agilizar la instalación básica.

3. **Directorio de Datos (.../data):**

Es el directorio crítico donde residen físicamente las tablas y archivos de la base de datos. Mantener la ruta clara o por defecto es vital para localizar fácilmente archivos a la hora de hacer respaldos o levantar bases de datos externas.

4. **Asignación de Contraseña para el Superusuario postgres:**

Definir una contraseña segura protege la cuenta principal de administración (postgres), la cual tiene control total sobre todos los recursos del motor.

5. **Configuración del Puerto (5432 o personalizado):**

Razones: El puerto estándar de PostgreSQL es el 5432. Mantener un valor definido asegura que el motor escuche correctamente las peticiones de los clientes. (*En el documento se ejemplifica el uso del puerto 40628 por motivos académicos/identificación de matrícula*).

6. **Configuración Regional (DEFAULT / es-MX):**

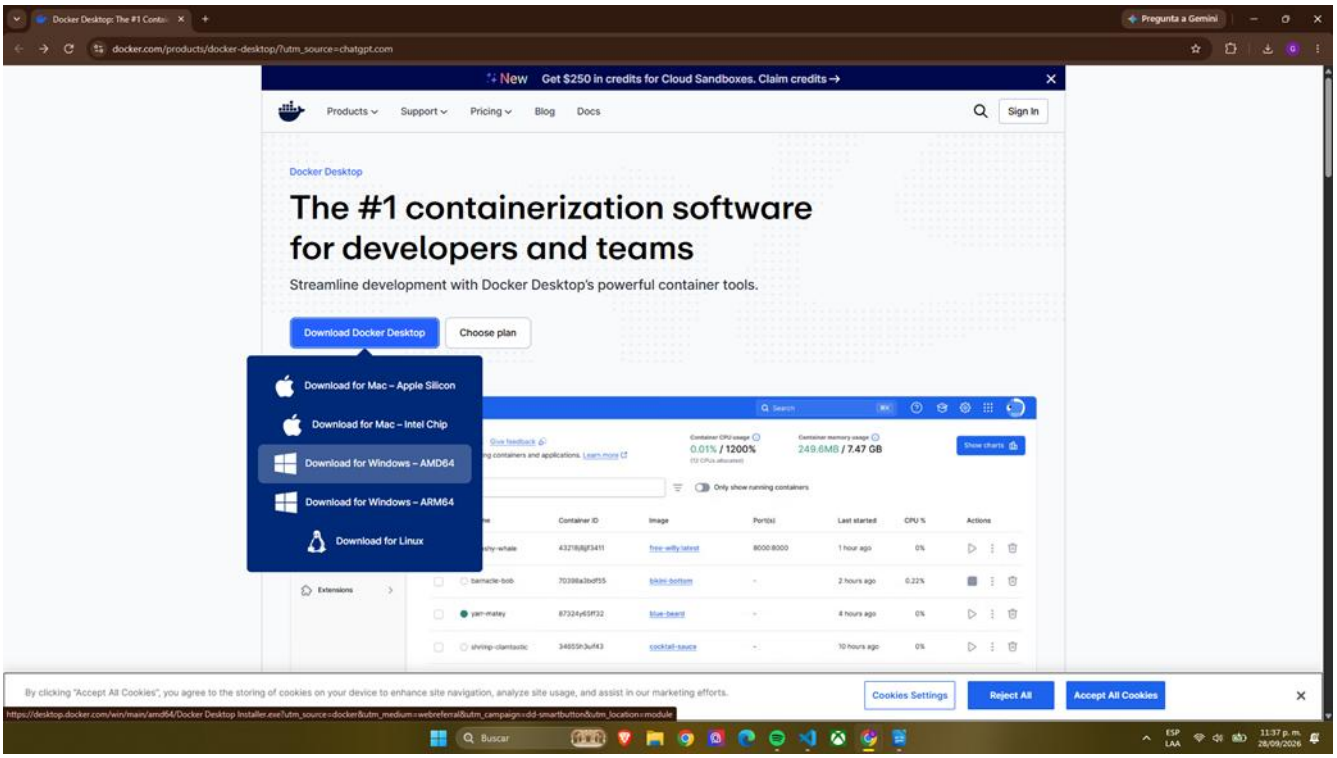
Al dejarlo en DEFAULT o definirlo en es-MX, se establece el ordenamiento alfabético (*collation*), la compatibilidad de caracteres y los formatos de datos como las fechas (DD/MM/AAAA frente a MM/DD/AAAA).

Instalación y Configuración de Docker Compuse

1. Instalación de Docker Desktop

El primer paso fue instalar Docker Desktop, aplicación que permite utilizar y administrar Docker desde Windows.

Docker Desktop proporciona las herramientas necesarias para trabajar con contenedores, imágenes, redes y volúmenes, además de permitir iniciar y administrar el motor de Docker. Una vez instalado, se abrió Docker Desktop y se esperó a que el motor de Docker estuviera completamente iniciado.



2. Preparación del proyecto

Posteriormente se creó una carpeta para almacenar los archivos de la práctica:

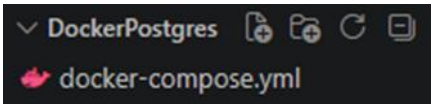
Documents/

└── DockerPostgres/

Dentro de esta carpeta se creó el archivo:

docker-compose.yml

El archivo fue creado y editado utilizando Visual Studio Code.



3. Configuración de Docker Compose

Para realizar la práctica se utilizó un archivo docker-compose.yml proporcionado por el profesor.

El archivo contiene la configuración necesaria para ejecutar los servicios de PostgreSQL y pgAdmin mediante Docker Compose.

De manera general, la configuración indica los servicios que deben ejecutarse, las imágenes que deben utilizarse, los puertos, volúmenes y la red que utilizarán los contenedores.

En esta práctica se utilizó la configuración proporcionada sin realizar modificaciones.

```yaml
services:

  postgres:
    image: postgres:17
    container_name: postgres_db
    restart: unless-stopped
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin123
      POSTGRES_DB: db_test
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - db_network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U admin -d db_test"]
      interval: 10s
      timeout: 5s
      retries: 5
  pgadmin:
    image: dpage/pgadmin4
    container_name: pgadmin_web
    restart: unless-stopped

    environment:
      PGADMIN_DEFAULT_EMAIL: admin@admin.com
      PGADMIN_DEFAULT_PASSWORD: admin123

    ports:
      - "8080:80"
```

```
    volumes:
      - pgadmin_data:/var/lib/pgadmin
    networks:
      - db_network
    depends_on:
      postgres:
        condition: service_healthy

volumes:
  postgres_data:
    name: postgres_data

  pgadmin_data:
    name: pgadmin_data

networks:
  db_network:
    name: postgres_network
    driver: bridge
```

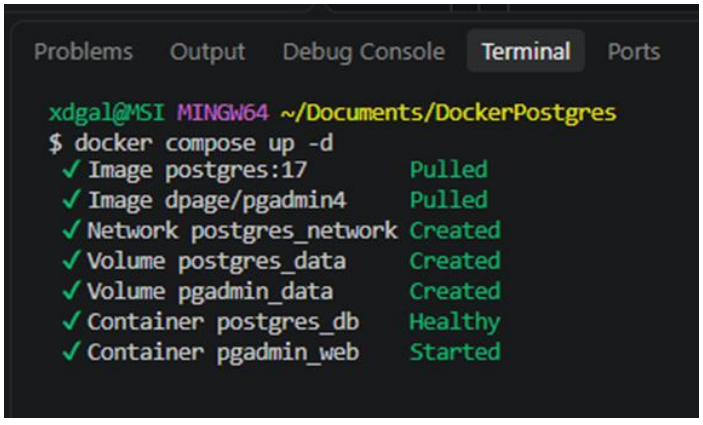
4. Ejecución desde la terminal

Con el archivo guardado, se abrió la terminal integrada de Visual Studio Code y se verificó que se estuviera trabajando dentro de la carpeta del proyecto:

~/Documents/DockerPostgres

Posteriormente se ejecutó el comando de Docker Compose proporcionado por el profesor para iniciar los servicios.

Este proceso hizo que Docker leyera el archivo de configuración, descargara las imágenes necesarias en caso de no encontrarse instaladas, creara los contenedores y los iniciara.



docker compose up -d

Inicia los servicios definidos en el archivo docker-compose.yml en segundo plano. Docker descargó las imágenes de PostgreSQL y pgAdmin, creó la red y los volúmenes necesarios, y posteriormente creó e inició los contenedores postgres_db y pgadmin_web. El estado Healthy de PostgreSQL indica que el servicio quedó listo para aceptar conexiones.

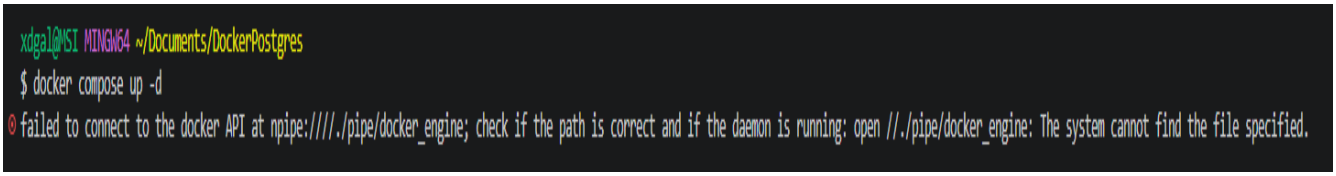
5. Problema inicial con Docker Engine

Al realizar el primer intento de ejecución desde la terminal se presentó un error relacionado con docker_engine.

El mensaje indicaba que Docker no podía conectarse con el motor de Docker.

Esto ocurrió porque Docker Desktop todavía no estaba iniciado correctamente.

Después de abrir Docker Desktop y esperar a que el motor estuviera disponible, se volvió a ejecutar el comando de Docker Compose y el proceso pudo continuar correctamente.



6. Comprobación de los contenedores

Una vez ejecutado Docker Compose, se comprobó desde la terminal que los contenedores estuvieran funcionando.

Se verificaron los servicios correspondientes a:

- PostgreSQL
- pgAdmin

Esto permitió confirmar que Docker había leído correctamente la configuración y había iniciado los servicios.

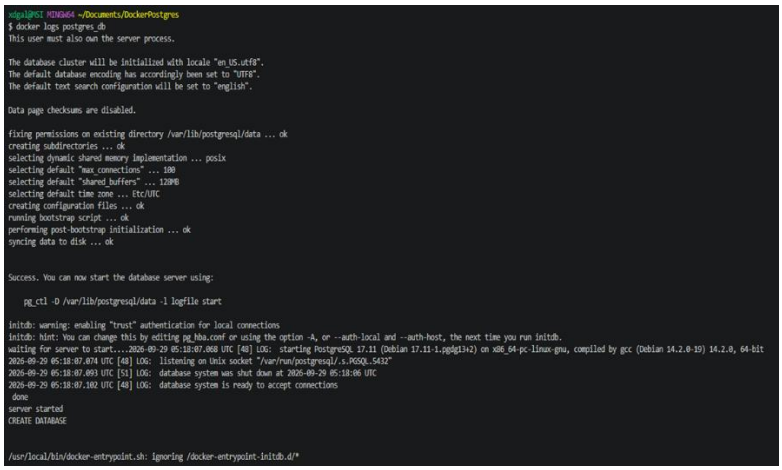


docker compose ps

Muestra el estado de los servicios administrados por Docker Compose. En este caso permitió comprobar que los contenedores postgres_db y pgadmin_web se encontraban activos, además de mostrar los puertos utilizados por cada servicio.

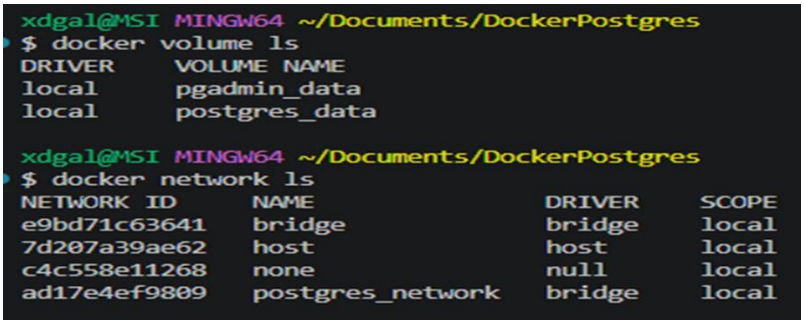
docker ps

Muestra los contenedores que se encuentran actualmente en ejecución en Docker. El resultado permitió comprobar directamente que postgres_db y pgadmin_web estaban funcionando y que sus puertos estaban correctamente asignados: 5432 para PostgreSQL y 8080 para pgAdmin.



docker logs postgres_db

Muestra los registros de funcionamiento del contenedor de PostgreSQL. Los mensajes permitieron comprobar el proceso de inicialización de la base de datos y confirmar que PostgreSQL terminó correctamente su configuración y quedó listo para aceptar conexiones en el puerto 5432.



docker volume ls

Muestra los volúmenes de almacenamiento administrados por Docker. El resultado permitió comprobar que se crearon postgres_data y pgadmin_data, utilizados para conservar los datos de PostgreSQL y pgAdmin.

docker network ls

Muestra las redes disponibles en Docker. El resultado permitió comprobar la creación de postgres_network, la red utilizada para permitir la comunicación entre los servicios de PostgreSQL y pgAdmin.

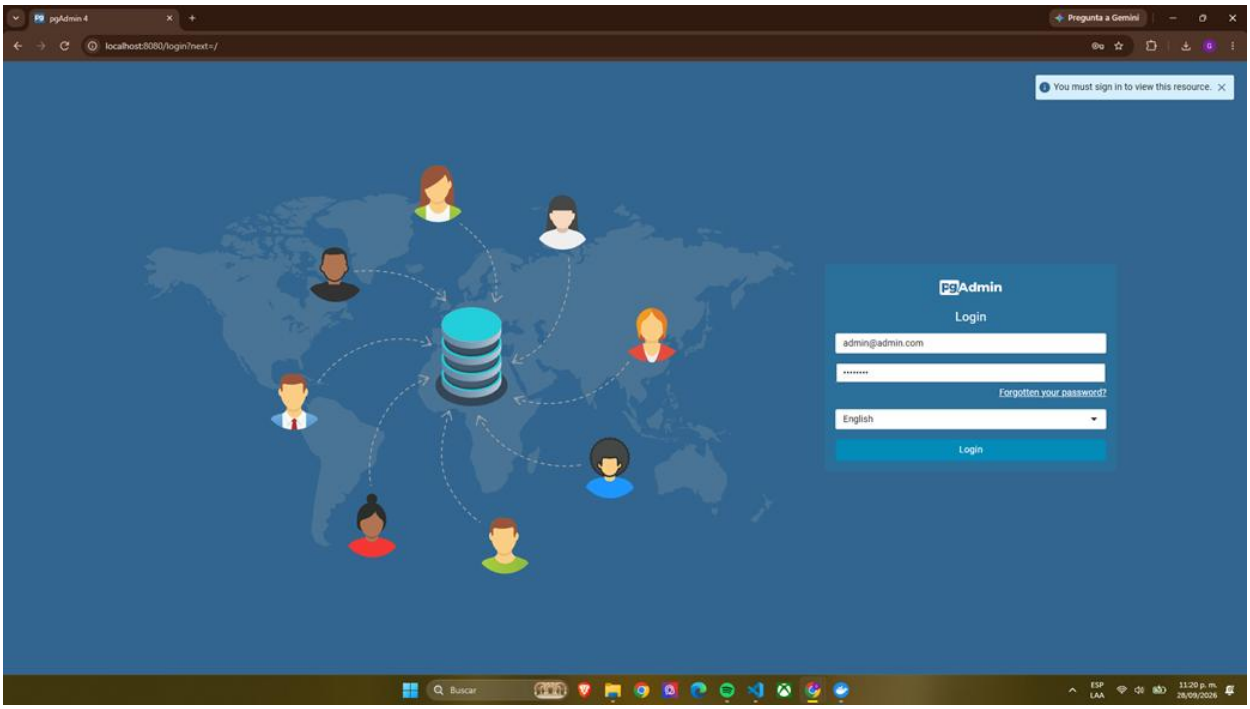
7. Acceso a pgAdmin

Después de iniciar los contenedores, se abrió un navegador web y se accedió a:

http://localhost:8080

Esto permitió abrir la interfaz web de pgAdmin, que se encontraba ejecutándose dentro de uno de los contenedores.

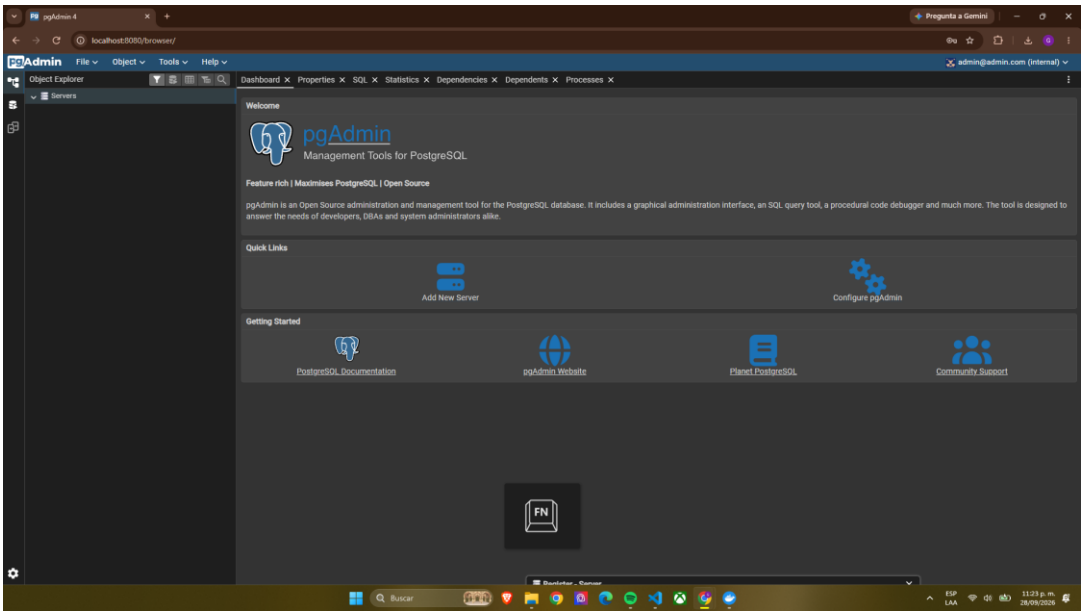
Se utilizaron las credenciales establecidas en la configuración proporcionada por el profesor para acceder a la aplicación.

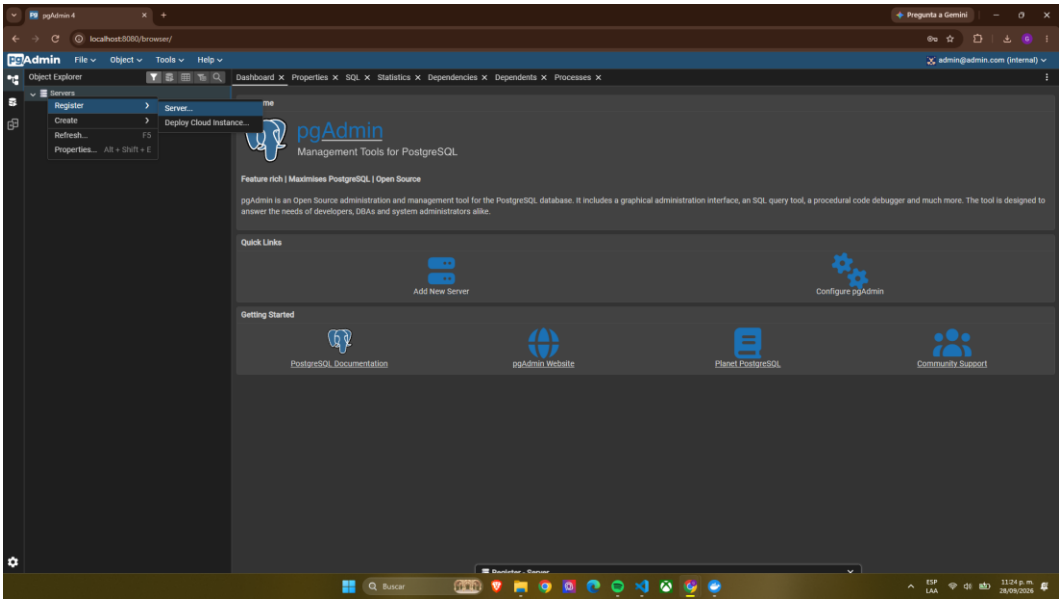


8. Comprobación de PostgreSQL

Finalmente se comprobó el funcionamiento de PostgreSQL mediante pgAdmin.

De esta manera se pudo verificar que el servicio de base de datos estaba ejecutándose correctamente y que podía ser administrado mediante la interfaz gráfica de pgAdmin.





Conclusión del Manual

El presente documento demuestra con éxito la instalación, configuración y verificación del sistema de gestión de bases de datos **PostgreSQL** junto con su herramienta de administración **pgAdmin 4**, abordando dos metodologías distintas pero complementarias: la **instalación nativa** sobre Windows 11 y el **despliegue contenedorizado** mediante Docker Compose.

A partir del desarrollo de este manual se destacaron las siguientes cosas:

- **Comprensión de Parámetros del Sistema (Instalación Nativa):** La instalación directa permitió analizar el impacto técnico de cada parámetro de configuración, tales como la asignación de la ruta de almacenamiento de datos, la gestión del superusuario postgres, la selección de puertos de escucha y la definición de la configuración regional (locale), factores clave para el rendimiento y la seguridad del servidor.

- **Ventajas de la Contenedorización (Docker Compose):** El uso de Docker facilitó un entorno aislado, reproducible y portátil. A través del archivo docker-compose.yml, se logró instanciar tanto PostgreSQL como pgAdmin en pocos pasos, aprovechando el uso de **volúmenes persistentes** (postgres_data y pgadmin_data) para la protección de la información y una **red virtual dedicada** (postgres_network) para la comunicación segura entre contenedores.

- **Diagnóstico y Verificación Operativa:** Se puso en práctica el monitoreo del estado de los servicios mediante comandos de terminal (docker ps, docker logs, docker volume ls), así como la resolución de problemas comunes en entornos de desarrollo, como la correcta inicialización del motor de Docker (*Docker Engine*) previo al despliegue.